

Measurement Systems

Lab Report 6

Instructor: Dr. Ali Sadighi

Author:
Roozbeh Amini - 810601291

February 19, 2026

Abstract

This laboratory session documents the design, calibration, and verification of a complete NTC thermistor temperature measurement system built around a 10 kΩ NTC thermistor, a unity-gain difference amplifier, and an Arduino Uno. The calibration strategy was constrained by the available reference instrument: a medical thermometer whose valid reading range spans only 32–43 °C, forcing all intermediate calibration points into the body-temperature region. The 0 °C endpoint was covered by an ice bath (thermodynamic fixed point), and the ≈90 °C endpoint by boiling water (temperature reduced from 100 °C by Tehran's altitude and post-boil cooling). Eight calibration points were collected at 50 Hz over 60-second windows; the last 10 seconds of each record were averaged to extract steady-state resistance.

The ice bath measurement established $V_{\text{th}}(0\,^\circ\text{C}) = 3.73\,\text{V}$ as the difference amplifier reference (Figure 8). After fitting the β-parameter model ($\beta = 3578.29\,\text{K}$, $R_0 = 9835.46\,\Omega$), the model was queried at 100 °C to extrapolate $V_{\text{th}}(100\,^\circ\text{C}) = 0.405\,\text{V}$ and hence $V_{\text{out}}(100\,^\circ\text{C}) = 3.325\,\text{V}$. This value was adopted as the Arduino external ADC reference ($V_{\text{AREF}} = 3.325\,\text{V}$), mapping ADC full scale to the nominal 0–100 °C measurement range and achieving a voltage resolution of 3.25 mV/step — a 33.5% improvement over the default 5 V reference. Input impedance matching was verified: the 1 MΩ difference amplifier inputs exceed the worst-case source impedance (7 468 Ω) by a factor of 13.4. Final verification returned +0.44 °C at the ice bath and 34.32 °C against a medical thermometer reading of 34.4 °C — a discrepancy of only 0.1 °C.

Contents

1 Introduction **5**
1.1 Session Objectives 5

2 Theoretical Background **5**
2.1 NTC Thermistor Characteristics 5
2.2 The β-Parameter Equation 6
2.2.1 Linearisation for Least-Squares Fitting 6
2.3 Voltage Divider 7
2.3.1 General Principle 7
2.3.2 Thermistor Voltage Divider 7
2.3.3 Potentiometer as an Adjustable Voltage Divider 7
2.3.4 Impedance Loading and the 10× Rule 8
2.4 Wheatstone Bridge 8
2.5 Difference Amplifier 9
2.6 Instrumentation Amplifier 9
2.7 ADC Reference Voltage and Resolution 10

3 Thermistor Calibration **10**
3.1 Calibration Strategy and Instrument Constraints 10
3.2 Reference Temperature Sources 11
3.3 Data Collection: 50 Hz Arduino Logger 12
3.4 Steady-State Analysis in MATLAB 13
3.5 Calibration Dataset 13
3.6 Steady-State Acquisition Plots and Observations 13
3.7 β-Model Fitting 19
3.8 Fitted Model Parameters and Calibration Curve 19
3.9 Step-by-Step Derivation of Key Design Voltages 20
3.9.1 Step 1 — Ice Bath $\Rightarrow V_{\text{ref}} = 3.73\,\text{V}$ 20
3.9.2 Step 2 — Model Extrapolation to 100 °C $\Rightarrow V_{\text{AREF}} = 3.325\,\text{V}$ 20
3.9.3 Voltage Table at All Calibration Points 21

4 Circuit Design and Signal Conditioning **22**
4.1 System Overview 22
4.2 Stage 1: Thermistor Voltage Divider 22
4.2.1 Circuit Description 22
4.2.2 Wiring on the Breadboard 22
4.3 Stage 2: Reference Voltage Generator (3.73 V) 23
4.3.1 Circuit Description 23
4.3.2 Wiring on the Breadboard 23
4.3.3 Setting the Potentiometer with the Live Voltage Reader 23
4.4 Stage 3: Unity-Gain Difference Amplifier 24
4.4.1 Circuit Description and Pin Assignment 24
4.4.2 Wiring on the Breadboard 24
4.4.3 Why 1 MΩ Resistors 25
4.4.4 Verification of the 10× Impedance Rule 26
4.4.5 Quantitative Loading Error at 0 °C 26

4.4.6 Why the Output Was Not Amplified to 0–5 V 26
4.5 Stage 4: External ADC Reference (3.325 V) 27
4.5.1 Circuit Description 27
4.5.2 Wiring on the Breadboard 27
4.6 Full Assembled Circuit 28

5 Firmware Implementation **30**
5.1 AREF Safety Requirement 30
5.2 Signal-Chain Inversion Logic 30
5.3 Firmware Constants and Their Origins 30

6 Results and Verification **31**
6.1 Verification Procedure 31
6.2 Verification Results — Thermometer Agreement 31
6.3 Summary of Verification Accuracy 31

7 Discussion **32**
7.1 Model Validity and Calibration Density 32
7.2 ADC Resolution Improvement 32
7.3 Error Sources and Analysis 33

8 Conclusion **34**

Acknowledgements **35**

Appendices **37**

A Arduino Calibration Logger (50 Hz, 60 s) **37**

B Live Voltage and Resistance Reader **37**

C MATLAB Calibration Acquisition Script **38**

D MATLAB Model Fitting and Characteristic Plots **40**

E Final Temperature Measurement Firmware **42**

List of Figures

1 Typical NTC thermistor resistance vs. temperature characteristic [1]. . . . 5
2 The 10 kΩ NTC thermistor used in this experiment. 6
3 Voltage divider schematic: two series resistors between V_{cc} and GND produce a node voltage proportional to the lower resistance. 7
4 Wheatstone bridge schematic [1]. At balance the differential voltage $V_D = 0$. In the unbalanced configuration used in sensor applications, V_D is proportional to the deviation of R_x from its nominal value. 8
5 Difference amplifier schematic [1]. With all four resistors equal the output is $V_{\text{out}} = V_2 - V_1$. 9
6 Three-op-amp instrumentation amplifier topology [1]. Gain is set by the single resistor R_G. 9
7 Calibration setup. Both sensors were held in the bath until the medical thermometer displayed a stable reading before the 60-second acquisition was started. 12
8 60-second acquisition at 0 °C (ice bath). Rising transient reflects thermistor cooling from room temperature. Steady-state mean: $V_{\text{th}} = 3.7300\,\text{V}$, $R_{\text{th}} = 29{,}498.05\,\Omega$. 14
9 60-second acquisition at 33.2 °C. Steady-state: $R_{\text{th}} = 7{,}241.27\,\Omega$. 15
10 60-second acquisition at 34.4 °C. Steady-state: $R_{\text{th}} = 6{,}832.37\,\Omega$. 15
11 60-second acquisition at 35.3 °C. Steady-state: $R_{\text{th}} = 6{,}585.11\,\Omega$. 16
12 60-second acquisition at 37.0 °C. Steady-state: $R_{\text{th}} = 6{,}135.16\,\Omega$. 16
13 60-second acquisition at 37.8 °C. Steady-state: $R_{\text{th}} = 5{,}944.50\,\Omega$. 17
14 60-second acquisition at 38.2 °C. Steady-state: $R_{\text{th}} = 5{,}892.11\,\Omega$. 17
15 60-second acquisition at 90 °C (boiling water). The sharp initial drop and slow cooling drift are clearly visible. Steady-state (last 10 s): $V_{\text{th}} = 0.5100\,\text{V}$, $R_{\text{th}} = 1{,}135.84\,\Omega$. 18
16 Calibration curve: measured data points (red circles) and fitted β-model (blue line). The model is well-constrained near 0 °C and the 33–38 °C cluster, and extrapolates to 100 °C to determine V_{AREF}. 19
17 Expected thermistor characteristics from the calibrated model over 0–100 °C. *Left:* resistance $R(T)$. *Right:* voltage divider output $V_{\text{th}}(T)$ for $V_{cc} = 5\,\text{V}$, $R_1 = 10\,\text{k}\Omega$. Calibration points are overlaid on both panels. 20
18 Stage 1 thermistor voltage divider wiring. The V_{th} node connects via jumper to the inverting input of Stage 3. 23
19 Stage 3 difference amplifier wiring. All four resistors are 1 MΩ. Inverting input: V_{th} from Stage 1. Non-inverting input: $V_{\text{ref}} = 3.73\,\text{V}$ from Stage 2. Output to Arduino A0. 25
20 Stage 4 AREF wiring. The 5 kΩ protection resistor is mandatory between the potentiometer wiper and the AREF pin. 27
21 Complete measurement circuit: schematic (left) and physical implementation (right). The stage-to-stage signal flow follows the schematic left-to-right order on the breadboard. A full walkthrough video is available at [4]. 29

1. Introduction

NTC (Negative Temperature Coefficient) thermistors are widely used in mechatronics instrumentation owing to their high sensitivity, low cost, and compact form factor [1]. Their strongly nonlinear resistance–temperature characteristic demands a physics-based calibration model and careful signal conditioning before the output can be connected to a microcontroller ADC.

This report follows the complete laboratory workflow prescribed by the course manual [1]: thermistor characterisation, signal conditioning circuit design and impedance analysis, MATLAB-based model fitting, and deployment of calibrated real-time firmware to an Arduino Uno. A key organisational decision is to **present calibration first**: every circuit design parameter — the difference amplifier reference voltage $V_{\text{ref}} = 3.73\,\text{V}$ and the external ADC reference $V_{\text{AREF}} = 3.325\,\text{V}$ — is derived from calibration measurements and model extrapolation; neither can be determined from the thermistor datasheet alone.

1.1. Session Objectives

1. Understand the operating principles of NTC thermistors and the β-parameter model.
2. Design and build a voltage divider, difference amplifier, and external ADC reference for thermistor signal conditioning.
3. Collect calibration points, fit the β model by linearised least-squares regression in MATLAB, and use the model to determine the ADC reference voltage.
4. Deploy calibrated temperature-reporting firmware to the Arduino Uno and verify the system as a functional thermometer.

2. Theoretical Background

2.1. NTC Thermistor Characteristics

An NTC thermistor is a semiconductor resistor whose resistance decreases sharply as temperature rises [1]. The mechanism is band-gap conduction: at higher temperatures more electrons are thermally promoted into the conduction band, increasing conductivity and reducing resistance. This makes NTC thermistors highly sensitive but strongly nonlinear, as shown in Figure 1. The thermistor used in this experiment (Figure 2) has a nominal resistance of $10\,\text{k}\Omega$ at $25\,^{\circ}\text{C}$.

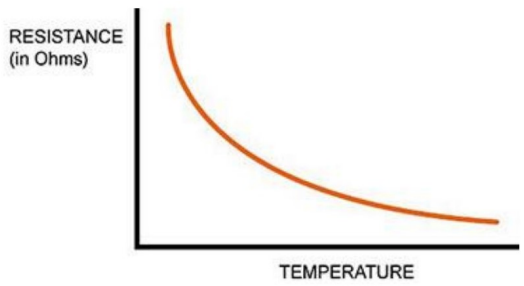


Figure 1: Typical NTC thermistor resistance vs. temperature characteristic [1].



Figure 2: The 10 kΩ NTC thermistor used in this experiment.

2.2. The β-Parameter Equation

The lab manual [1] specifies the β-parameter equation for NTC characterisation:

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{\beta} \ln \frac{R}{R_0} \tag{1}$$

Solving for resistance:

$$\boxed{R(T) = R_0 \exp\left[\beta\left(\frac{1}{T} - \frac{1}{T_0}\right)\right]} \tag{2}$$

where $T_0 = 298.15\,\mathrm{K}$ (25 °C) is the standard reference, R_0 is the resistance at T_0, and β is a material constant (3000–4000 K for typical NTC types). Inverting for temperature output:

$$T = \frac{1}{\dfrac{1}{T_0} + \dfrac{1}{\beta} \ln \dfrac{R}{R_0}} \tag{3}$$

2.2.1. Linearisation for Least-Squares Fitting

Taking the logarithm of Equation (2):

$$\underbrace{\ln R}_{y} = \underbrace{\ln R_0}_{\text{intercept}} + \beta \underbrace{\left(\frac{1}{T} - \frac{1}{T_0}\right)}_{x} \tag{4}$$

Substituting $x_i = 1/T_i - 1/T_0$ and $y_i = \ln R_i$ yields the standard linear model $y = \beta\, x + \ln R_0$. MATLAB's `polyfit(x,y,1)` returns slope $\hat{\beta}$ and intercept $\ln \hat{R}_0$ by ordinary least squares, with $\hat{R}_0 = \exp(\text{intercept})$.

2.3. Voltage Divider

2.3.1. General Principle

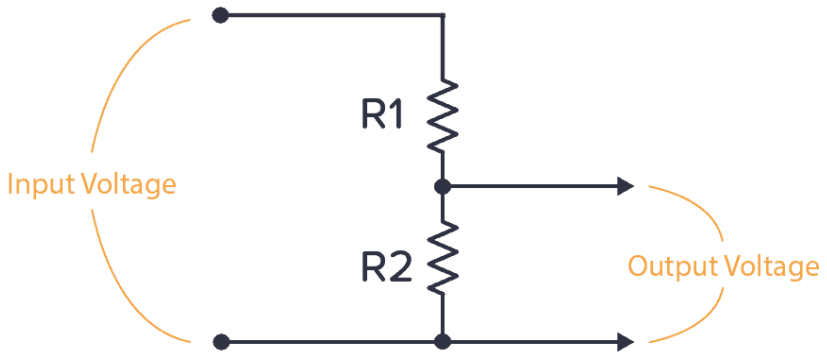


Figure 3: Voltage divider schematic: two series resistors between V_{cc} and GND produce a node voltage proportional to the lower resistance.

As shown in Figure 3, two series resistors divide the supply voltage:

$$V_{\text{out}} = V_{cc} \cdot \frac{R_2}{R_1 + R_2} \tag{5}$$

Any load connected to the output node forms a parallel combination with R_2, reducing the effective lower resistance and pulling V_{out} below its ideal value. This *loading* effect is quantified in Section 2.3.4.

2.3.2. Thermistor Voltage Divider

With the thermistor as the lower element ($R_2 = R_{\text{th}}$, $R_1 = 10\,\text{k}\Omega$, $V_{cc} = 5\,\text{V}$):

$$V_{\text{th}} = V_{cc} \cdot \frac{R_{\text{th}}}{R_1 + R_{\text{th}}}, \qquad R_{\text{th}} = R_1 \cdot \frac{V_{\text{th}}}{V_{cc} - V_{\text{th}}} \tag{6}$$

Because NTC resistance decreases with temperature, V_{th} also decreases. At 0 °C ($R_{\text{th}} \approx 29{,}500\,\Omega$) the node sits near 3.73 V; at 100 °C ($R_{\text{th}} \approx 881\,\Omega$, from model) it falls to 0.405 V.

2.3.3. Potentiometer as an Adjustable Voltage Divider

A potentiometer divides its total resistance R_{total} into an upper portion R_{top} and a lower portion R_{bot}. With end terminals at V_{cc} and GND, the wiper voltage is:

$$V_{\text{wiper}} = V_{cc} \cdot \frac{R_{\text{bot}}}{R_{\text{total}}} \tag{7}$$

Turning the adjustment screw changes R_{bot} and sweeps V_{wiper} continuously from 0 to V_{cc}. This property was used to set both the 3.73 V difference amplifier reference (potentiometer 1) and the 3.325 V ADC reference (potentiometer 2), each verified using a live voltage reader sketch on the Arduino.

2.3.4. Impedance Loading and the 10× Rule

The Thévenin output impedance of the voltage divider is:

$$R_{\mathrm{Th}} = R_1 \parallel R_{\mathrm{th}} = \frac{R_1 R_{\mathrm{th}}}{R_1 + R_{\mathrm{th}}} \tag{8}$$

The first-order loading voltage error when a load $R_L \gg R_{\mathrm{Th}}$:

$$\varepsilon_V \approx V_{\mathrm{th}} \cdot \frac{R_{\mathrm{Th}}}{R_L} \tag{9}$$

To keep this below ≈9%, the **10× rule** requires:

$$\boxed{R_L \geq 10\, R_{\mathrm{Th}}} \tag{10}$$

2.4. Wheatstone Bridge

The Wheatstone bridge, shown in Figure 4, measures an unknown resistance by balancing two legs of a four-arm diamond network [1]. At balance:

$$R_x = \frac{R_2}{R_1} \cdot R_3 \tag{11}$$

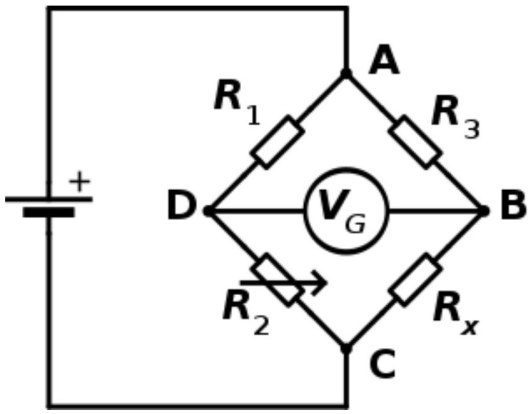


Figure 4: Wheatstone bridge schematic [1]. At balance the differential voltage $V_D = 0$. In the unbalanced configuration used in sensor applications, V_D is proportional to the deviation of R_x from its nominal value.

In an unbalanced configuration the differential midpoint voltage is proportional to the deviation of R_x from a reference. The circuit in this experiment achieves an equivalent differential measurement by subtracting a fixed reference voltage from V_{th} using a difference amplifier, without requiring a physical four-arm bridge.

2.5. Difference Amplifier

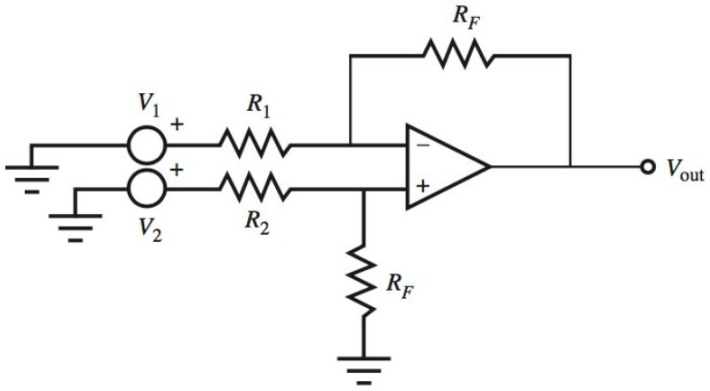


Figure 5: Difference amplifier schematic [1]. With all four resistors equal the output is $V_{\text{out}} = V_2 - V_1$.

The difference amplifier (Figure 5) subtracts two voltages with a gain set by the resistor ratio [1]:

$$V_{\text{out}} = \frac{R_F}{R_{\text{in}}}(V_2 - V_1) \tag{12}$$

With all four resistors equal ($R_F = R_{\text{in}}$), gain is unity:

$$\boxed{V_{\text{out}} = V_2 - V_1} \tag{13}$$

In this project $V_2 = V_{\text{ref}} = 3.73\,\text{V}$ (potentiometer 1) and $V_1 = V_{\text{th}}$ (thermistor divider node). The output represents the departure of the thermistor voltage from the 0 °C ice bath level.

2.6. Instrumentation Amplifier

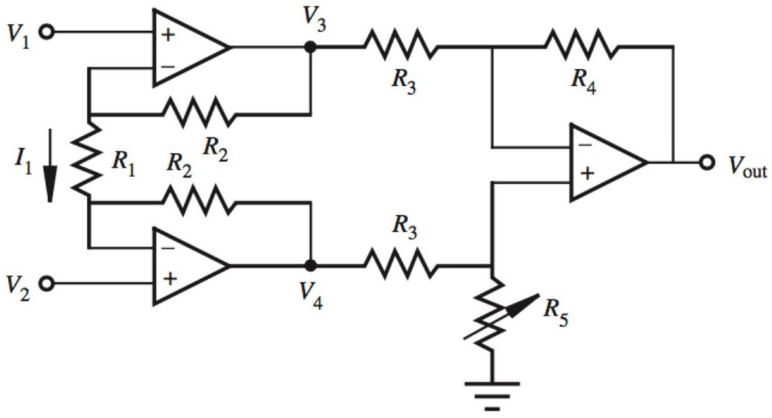


Figure 6: Three-op-amp instrumentation amplifier topology [1]. Gain is set by the single resistor R_G.

An instrumentation amplifier (Figure 6) includes integral input buffers giving near-infinite input impedance, eliminating any loading concern. Gain is set by a single resistor:

$$G = 1 + \frac{2R}{R_G} \tag{14}$$

Alternative Implementations

Instrumentation amplifier (e.g. AD620): Would eliminate the loading problem entirely and allow gain > 1 via a single external resistor.
Dual DC supply: With a $\pm 9\,\text{V}$ or $\pm 12\,\text{V}$ bench supply the op-amp output could swing freely across the full ADC range, making the external AREF potentiometer unnecessary. The reasons for not using a gain stage instead are discussed in Section 4.4.6.

2.7. ADC Reference Voltage and Resolution

The Arduino Uno's 10-bit ADC maps [0, V_{AREF}] to [0, 1023] [1]:

$$V_{\text{measured}} = \frac{\text{ADC}}{1023} \cdot V_{\text{AREF}}, \qquad \Delta V = \frac{V_{\text{AREF}}}{1023} \tag{15}$$

With the default 5 V reference and a signal spanning 0–3.325 V, only 680 of 1023 steps (66.5%) are utilised. Setting $V_{\text{AREF}} = 3.325\,\text{V}$ recovers all 1023 steps, giving 3.25 mV/step — a 33.5% improvement in voltage resolution compared to the default.

3. Thermistor Calibration

Calibration is presented before circuit design because every design parameter flows directly from calibration measurements. The difference amplifier reference $V_{\text{ref}} = 3.73\,\text{V}$ comes from the ice bath measurement; the ADC reference $V_{\text{AREF}} = 3.325\,\text{V}$ is obtained by querying the fitted β-model at 100 °C. Neither value can be read from the thermistor datasheet.

3.1. Calibration Strategy and Instrument Constraints

The calibration campaign was shaped by the available reference instrument: a standard digital medical thermometer. This device was chosen because it is a readily available, calibrated, and stable reference for body-temperature measurements. However, it imposes a fundamental **range limitation**: medical thermometers are designed exclusively for body temperature measurement and their valid display range is approximately **32 °C to 43 °C**. Outside this window the thermometer displays `Lo` (below 32 °C) or `Hi` (above 43 °C) and cannot be used as a reference.

This constraint directly explains the distribution of calibration points:

- **Intermediate points (33–38 °C):** All collected within the medical thermometer's valid range by mixing cold and hot water in different proportions. The six intermediate points are concentrated in this narrow window not by design but by necessity.

- **0 °C (ice bath):** Collected without the thermometer. The melting point of ice at atmospheric pressure is a thermodynamic fixed point and does not require an external reference.
- **≈90 °C (boiling water):** Collected without the thermometer — the 90 °C bath temperature exceeds the thermometer's valid range. The assigned temperature is justified on physical grounds in Section 3.2.

An ideal calibration dataset would contain points uniformly distributed over the full 0–100 °C range (e.g. at 0, 15, 30, 45, 60, 75, 90 °C). The instrument limitation makes this impossible without access to a calibrated thermocouple or platinum RTD. The consequences on model uncertainty are discussed in Section 7.1.

3.2. Reference Temperature Sources

1. **Ice bath — 0 °C (thermodynamic fixed point).** A well-stirred mixture of crushed ice and distilled water. The melting point of ice at atmospheric pressure is accurate to better than 0.05 °C and requires no reference instrument. This was the first measurement performed; its steady-state voltage reading ($V_{\text{th}} = 3.73\,\text{V}$, Figure 8) directly sets V_{ref} for the difference amplifier.
2. **Boiling water — assigned 90 °C.** Water was heated to boiling in an electric kettle, the thermistor probe was immediately inserted, and the 60-second acquisition started. The water temperature was assigned 90 °C (rather than 100 °C) for two physically well-founded reasons:
   - **Tehran's altitude and reduced atmospheric pressure.** At approximately 1200–1500 m above sea level atmospheric pressure is ≈0.86–0.88 atm. The boiling point of water decreases by ≈0.34 °C per 100 m, reducing it by 4–5 °C at this location.
   - **Rapid Newton's law cooling during acquisition.** Once removed from its heat source the water cools rapidly because the driving temperature difference ($T_{\text{water}} - T_{\text{room}} \approx 75\,^{\circ}\text{C}$) is large. By the time the 60-second steady-state tail is evaluated the water has settled at ≈90 °C, as confirmed by the stable plateau in Figure 15.

   The 90 °C point serves as a **high-end calibration anchor** for the β-model fit. The ADC reference voltage is *not* derived from this direct measurement; instead the fitted model is extrapolated to 100 °C as described in Section 3.9.
3. **Mixed baths — 33–38 °C.** Cold and hot water combined in various proportions, with the medical thermometer co-immersed. Only readings within the valid 32–43 °C window were accepted.



(a) Medical thermometer (valid range 32–43 °C).

(b) Thermistor and thermometer co-immersed.

Figure 7: Calibration setup. Both sensors were held in the bath until the medical thermometer displayed a stable reading before the 60-second acquisition was started.

3.3. Data Collection: 50 Hz Arduino Logger

For each calibration point a dedicated Arduino sketch sampled the thermistor voltage at 50 Hz (every 20 ms) for exactly 60 seconds and transmitted comma-separated data at 115200 baud. A `millis()`-based trigger was used rather than `delay()` to ensure precise, drift-free sample spacing:

Listing 1: Core 50 Hz acquisition loop. Resistance is computed on-chip from the voltage divider inversion (Eq. (6)).

```
while (millis() - t0 < 60000UL) {
  if (millis() >= nextSample) {
    nextSample += 20;                          // 20 ms -> 50 Hz
    float v    = analogRead(A0) * 5.0f / 1023.0f;
    float R_th = 10000.0f * v / (5.0f - v);   // divider inversion
    Serial.print(t_s, 4); Serial.print(",");
    Serial.print(v, 6);   Serial.print(",");
    Serial.println(R_th, 2);
  }
}
```

The full logger sketch is given in Appendix A.

3.4. Steady-State Analysis in MATLAB

The 3000-sample record was received in MATLAB via the `serialport` API. The **last 10 seconds** (500 samples) constitute the steady-state window: by this point any initial thermal transient has decayed and the thermistor has fully equilibrated to the bath temperature. The mean and standard deviation are:

Listing 2: MATLAB steady-state extraction: the last `steady_win` seconds are averaged.

```
ss_mask = T >= (max(T) - steady_win);  % last 10 s
v_mean  = mean(V(ss_mask));
r_mean  = mean(R(ss_mask));
r_std   = std(R(ss_mask));
```

Each acquisition is visualised with: the raw 50 Hz signal (grey), a 1-second moving average (coloured), the green steady-state window, and the mean as a labelled dashed line, so any unresolved transient is immediately visible before accepting a point.

3.5. Calibration Dataset

Table 1: Eight calibration points from measurement. R_{th} values are steady-state means from the last 10 seconds of each 60-second record.

Pt	T_{ref} (°C)	T (K)	R_{th} (Ω)	**Source**
1	0.0	273.15	29 498.05	Ice bath
2	33.2	306.35	7 241.27	Mixed + therm.
3	34.4	307.55	6 832.37	Mixed + therm.
4	35.3	308.45	6 585.11	Mixed + therm.
5	37.0	310.15	6 135.16	Mixed + therm.
6	37.8	310.95	5 944.50	Mixed + therm.
7	38.2	311.35	5 892.11	Mixed + therm.
8	90.0	363.15	1 135.84	Boiling water

3.6. Steady-State Acquisition Plots and Observations

Figures 8–15 show the 60-second acquisition records for all eight calibration points. Each figure shows the raw 50 Hz signal (grey), a 1-second moving average (coloured line), the steady-state window (green shading), and the steady-state mean (labelled dashed line).

Ice bath — 0 °C (Figure 8). The thermistor entered the ice bath from room temperature (≈25 °C). Since NTC resistance *increases* as temperature *falls*, the voltage V_{th} rises visibly during the first 20–25 s as the thermistor cools. The change is large: $V_{\text{th}}(25\,^{\circ}\text{C}) \approx 2.48$ V rising to $V_{\text{th}}(0\,^{\circ}\text{C}) \approx 3.73$ V — a swing of +1.25 V. The final 10-second window shows a stable, low-noise plateau. The steady-state mean is $V_{\text{th}} = 3.7300$ V, which is used directly as $V_{\text{ref}} = 3.73$ V in the difference amplifier (Step 1, Section 3.9).

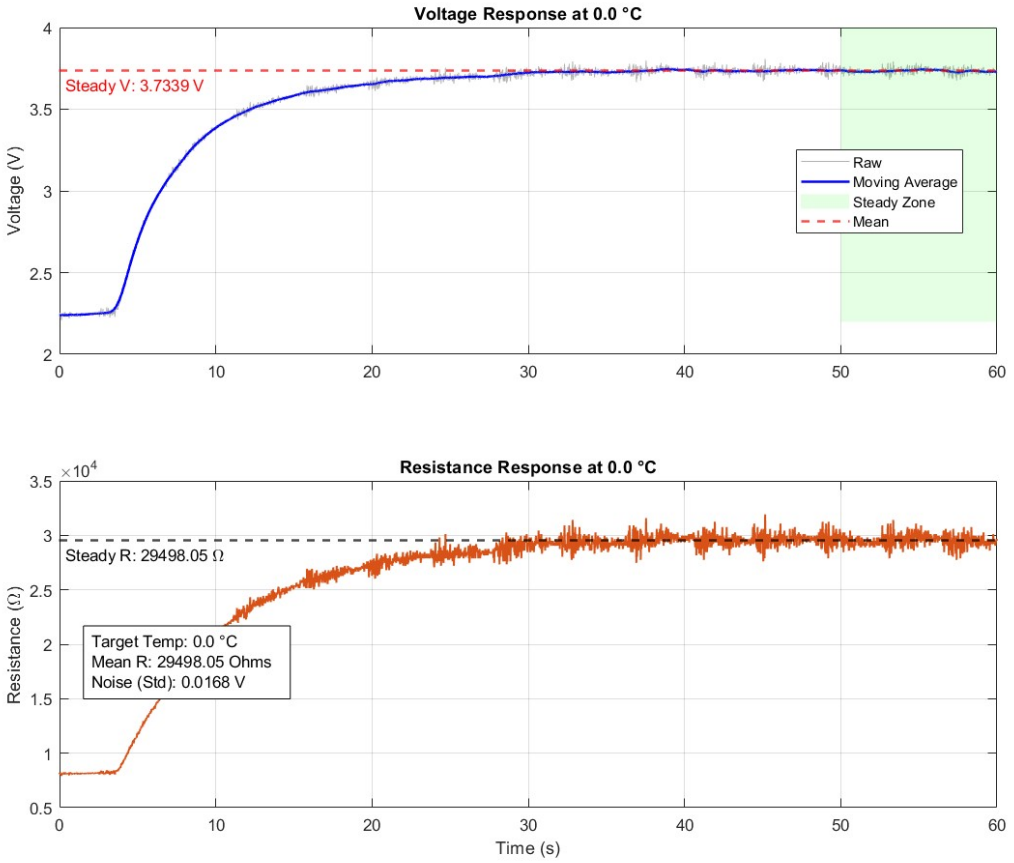


Figure 8: 60-second acquisition at 0 °C (ice bath). Rising transient reflects thermistor cooling from room temperature. Steady-state mean: $V_{\text{th}} = 3.7300\,\text{V}$, $R_{\text{th}} = 29{,}498.05\,\Omega$.

Mixed baths — 33–38 °C (Figures 9–14). These acquisitions show much faster equilibration than the ice bath because the temperature difference between the bath (33–38 °C) and the thermistor's starting temperature (≈25 °C) is small (8–13 °C vs 25 °C for the ice bath). The sensor reaches equilibrium within the first 5–10 s, leaving the remaining 50 s as a stable flat region. As temperature increases across the six baths, V_{th} decreases progressively from ≈2.10 V at 33.2 °C to ≈1.85 V at 38.2 °C, consistent with the NTC characteristic. The small standard deviation in each 10-second window confirms good thermal equilibration and a stable ADC signal.

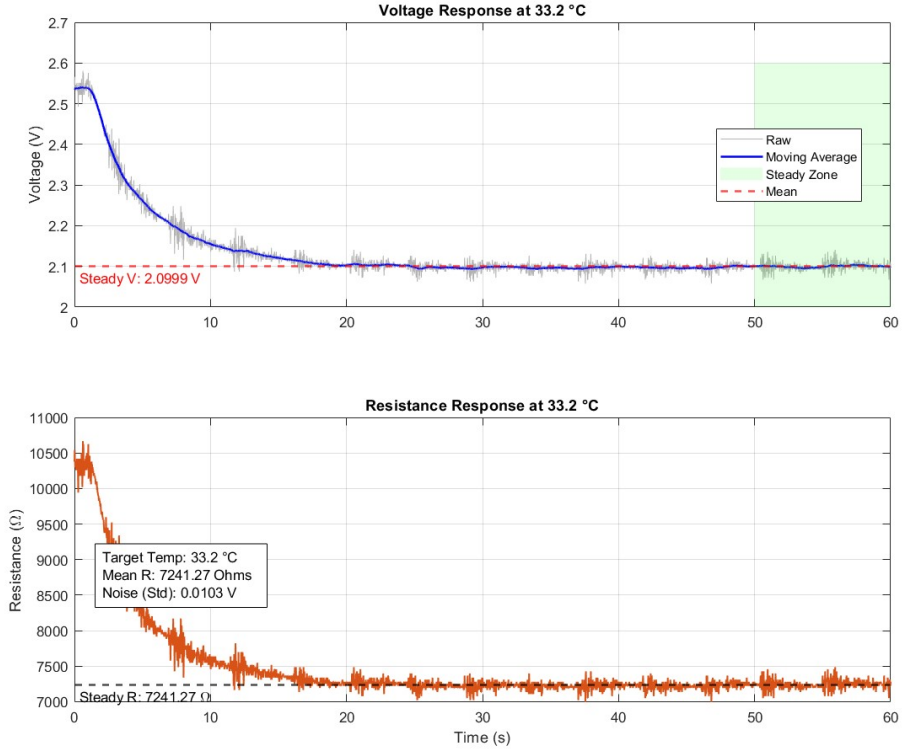


Figure 9: 60-second acquisition at 33.2 °C. Steady-state: $R_{\text{th}} = 7{,}241.27\,\Omega$.

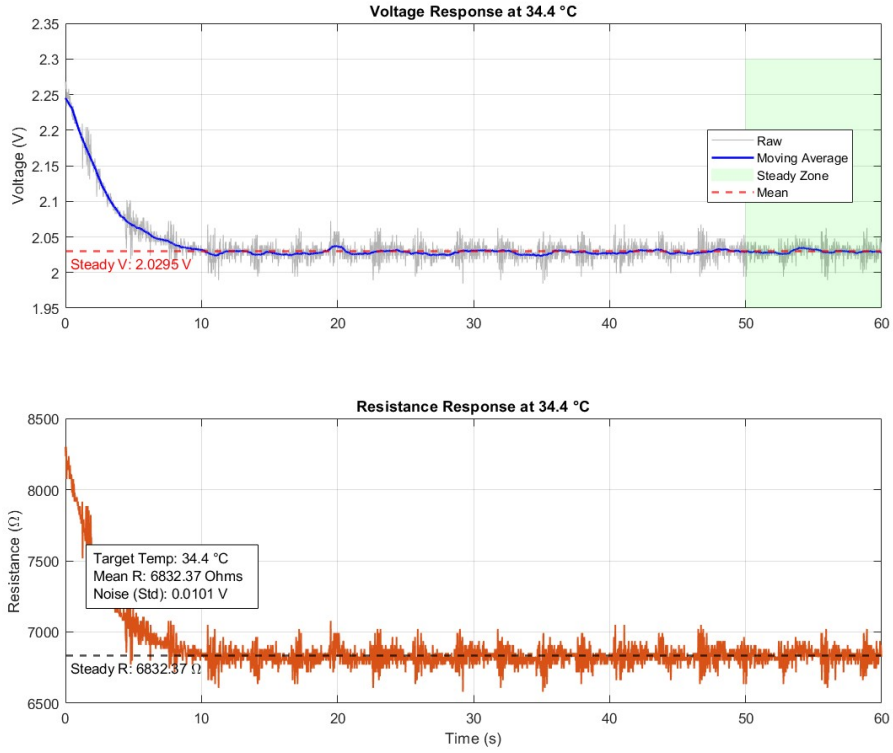


Figure 10: 60-second acquisition at 34.4 °C. Steady-state: $R_{\text{th}} = 6{,}832.37\,\Omega$.

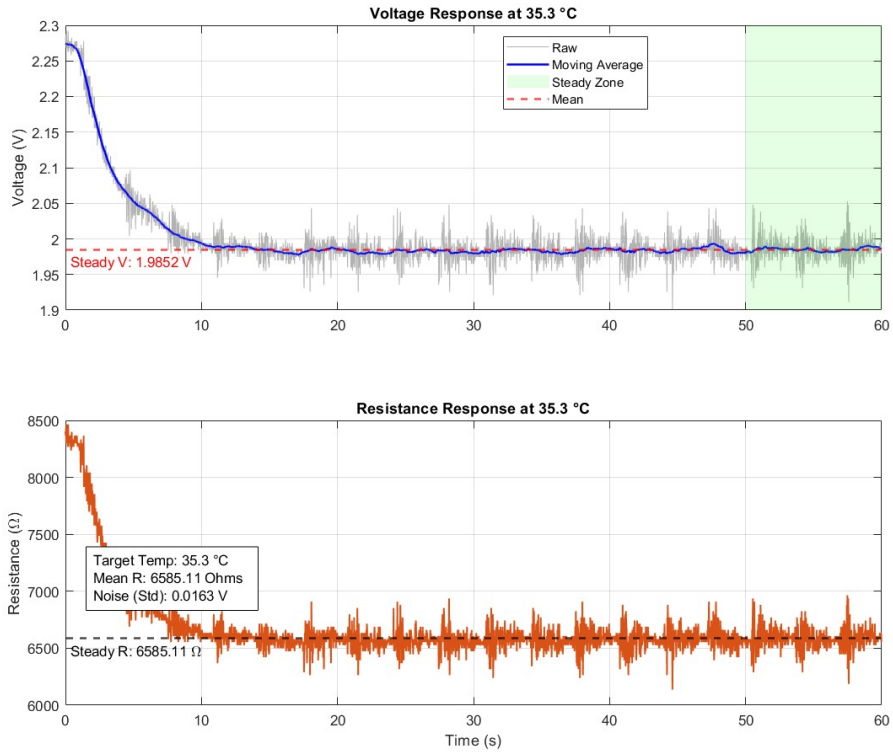


Figure 11: 60-second acquisition at 35.3 °C. Steady-state: $R_{\text{th}} = 6{,}585.11\,\Omega$.

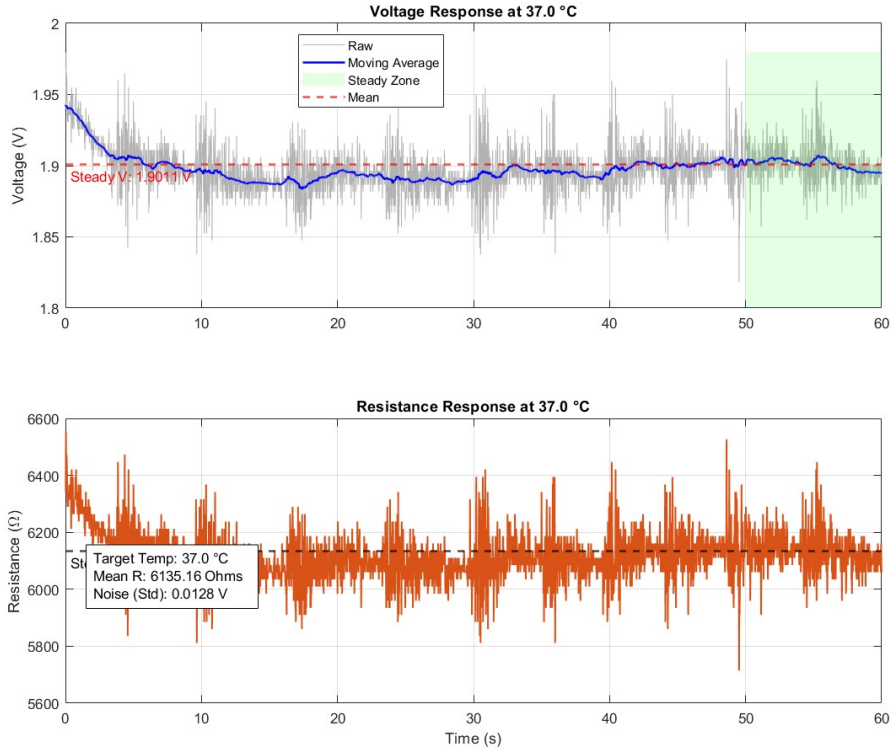


Figure 12: 60-second acquisition at 37.0 °C. Steady-state: $R_{\text{th}} = 6{,}135.16\,\Omega$.

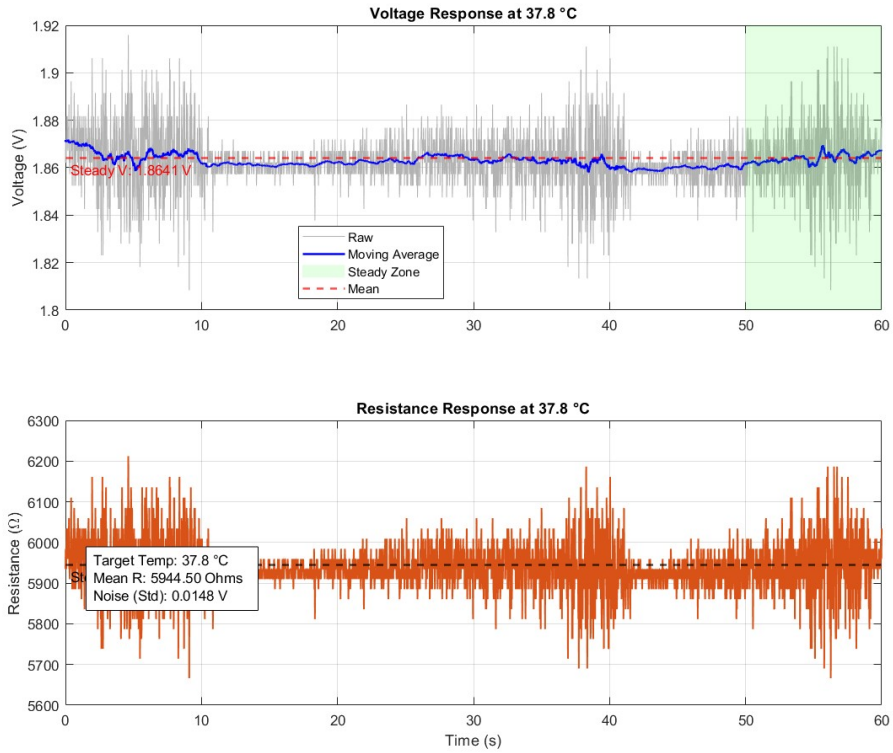


Figure 13: 60-second acquisition at 37.8 °C. Steady-state: $R_{\text{th}} = 5{,}944.50\,\Omega$.

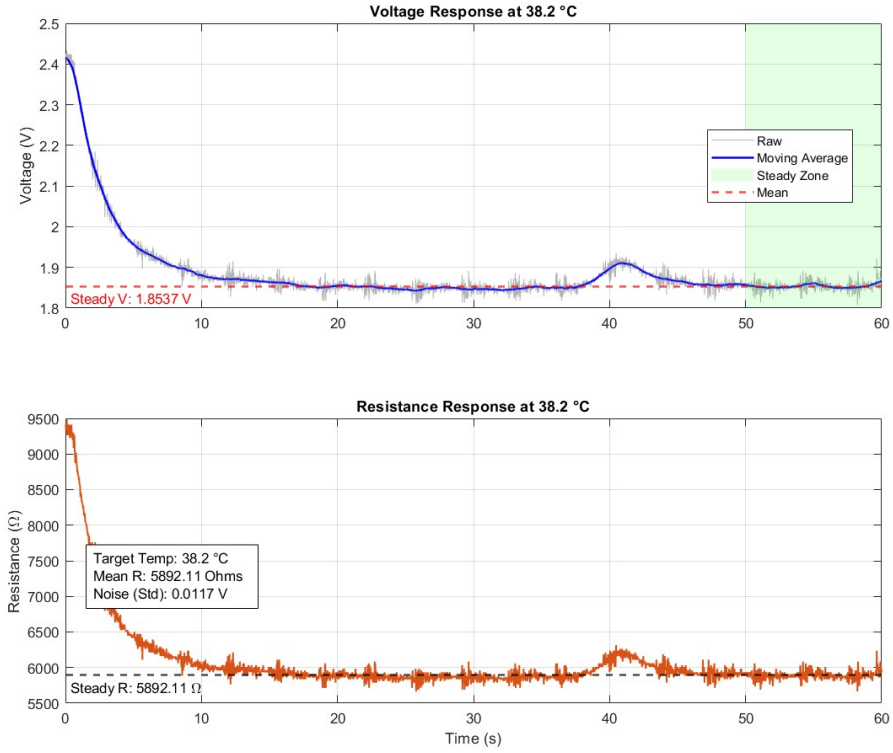


Figure 14: 60-second acquisition at 38.2 °C. Steady-state: $R_{\text{th}} = 5{,}892.11\,\Omega$.

Boiling water — 90 °C (Figure 15). This acquisition presents the most complex transient. The thermistor starts at room temperature and is suddenly immersed in very hot water. Resistance drops sharply in the first 5 s, pulling V_{th} steeply downward from ≈2.48 V toward its final value of ≈0.51 V — a drop of −1.96 V. Superimposed on this rapid equilibration is a *slow downward drift* caused by the water cooling (Newton's law): the large temperature driving force (≈75 °C above room) causes significant heat loss from the moment the kettle is unplugged. The final 10-second window shows a stable plateau confirming both the thermistor and the water temperature have settled. The steady-state mean is $V_{\text{th}} = 0.5100$ V and $R_{\text{th}} = 1{,}135.84\,\Omega$. This point is used as the high-end anchor of the β-model fit; the AREF voltage is subsequently derived by extrapolating the fitted model to 100 °C (Section 3.9).

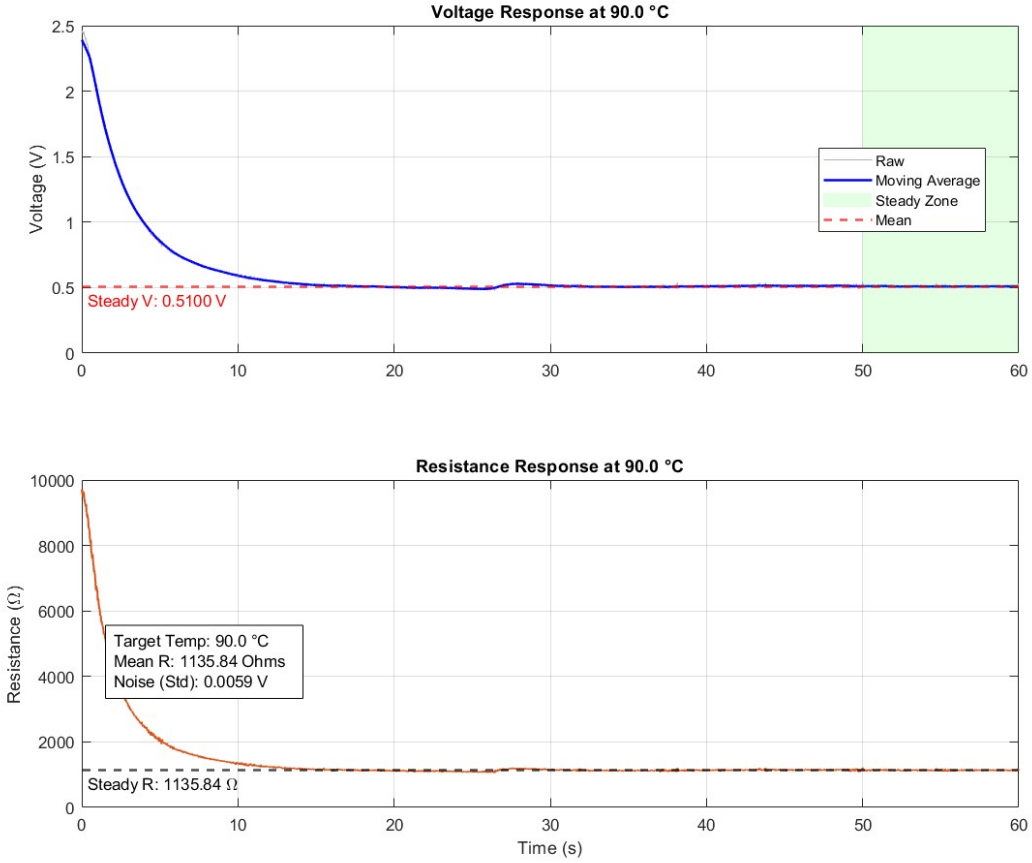


Figure 15: 60-second acquisition at 90 °C (boiling water). The sharp initial drop and slow cooling drift are clearly visible. Steady-state (last 10 s): $V_{\text{th}} = 0.5100$ V, $R_{\text{th}} = 1{,}135.84\,\Omega$.

3.7. β-Model Fitting

The calibration pairs $(T_i,\ R_i)$ from Table 1 are transformed to linearised coordinates and fitted in MATLAB:

Listing 3: Linearised β-model fitting.

```
X    = (1 ./ T_data_K) - (1 / T0);
Y    = log(R_data);
p    = polyfit(X, Y, 1);
Beta = p(1);
R0   = exp(p(2));
```

3.8. Fitted Model Parameters and Calibration Curve

$$\boxed{\beta = 3578.29\,\mathrm{K}, \qquad R_0 = 9835.46\,\Omega} \tag{16}$$

The fitted $\beta = 3578\,\mathrm{K}$ lies within the standard NTC range of 3000–4000 K, and $R_0 = 9835.46\,\Omega$ is within 1.65% of the nominal $10\,\mathrm{k}\Omega$ specification — both values are physically self-consistent. The calibration curve with all eight measured data points is shown in Figure 16.

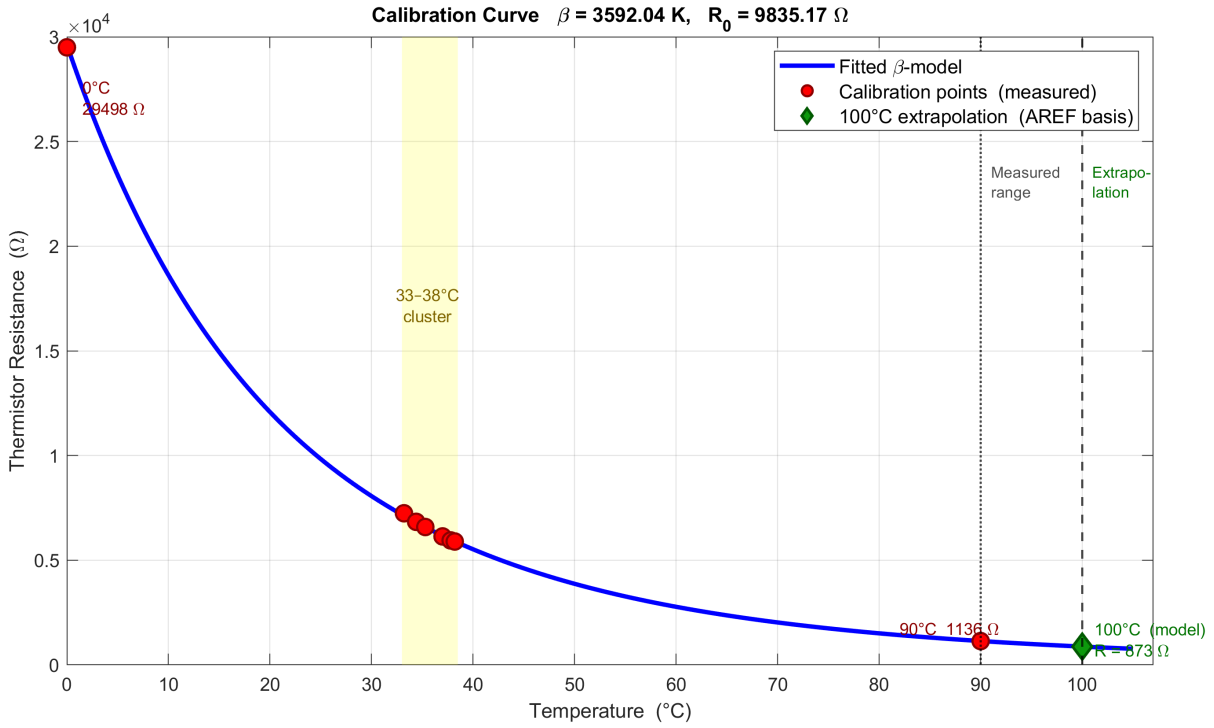


Figure 16: Calibration curve: measured data points (red circles) and fitted β-model (blue line). The model is well-constrained near 0 °C and the 33–38 °C cluster, and extrapolates to 100 °C to determine V_{AREF}.

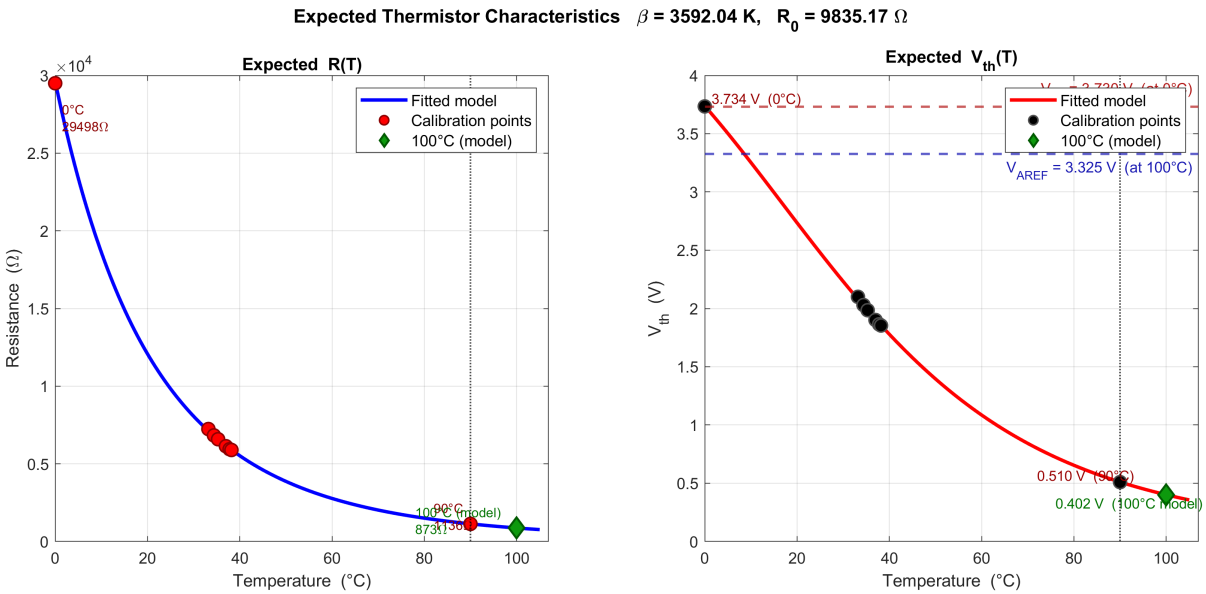


Figure 17: Expected thermistor characteristics from the calibrated model over 0–100 °C. *Left:* resistance $R(T)$. *Right:* voltage divider output $V_{\text{th}}(T)$ for $V_{cc} = 5\,\text{V}$, $R_1 = 10\,\text{k}\Omega$. Calibration points are overlaid on both panels.

The expected $R(T)$ and $V_{\text{th}}(T)$ characteristics derived from the calibrated model are shown in Figure 17. The exponential decay of resistance and the corresponding nonlinear voltage divider output are both well-matched by the eight calibration points, providing confidence in the model within the calibrated range.

3.9. Step-by-Step Derivation of Key Design Voltages

Both circuit design voltages are derived directly from calibration data and model output.

3.9.1. Step 1 — Ice Bath ⇒ $V_{\text{ref}} = 3.73\,\text{V}$

The stable plateau in Figure 8 gives $V_{\text{th}}(0\,^\circ\text{C}) = 3.7300\,\text{V}$. This can be confirmed analytically from the voltage divider (Equation (6)) using the measured steady-state resistance:

$$V_{\text{th}}(0\,^\circ\text{C}) = 5.0 \times \frac{29{,}498}{10{,}000 + 29{,}498} = 3.7341\,\text{V} \tag{17}$$

For the difference amplifier to output exactly 0 V at the ice bath reference the non-inverting input must match this voltage:

$$V_{\text{out}}(0\,^\circ\text{C}) = V_{\text{ref}} - V_{\text{th}}(0\,^\circ\text{C}) = 3.73 - 3.73 = 0\,\text{V} \tag{18}$$

Therefore potentiometer 1 was set to $V_{\text{ref}} = 3.73\,\text{V}$.

3.9.2. Step 2 — Model Extrapolation to 100 °C ⇒ $V_{\text{AREF}} = 3.325\,\text{V}$

Rather than deriving the ADC reference from the direct 90 °C measurement (which would limit the ADC mapping to the physically measured 0–90 °C range), the fitted β-model

was queried at the **nominal 100 °C** sea-level boiling point. This maps ADC full scale to a physically meaningful temperature endpoint.

Step 2a: Resistance at 100 °C from the model.

Substituting $T = 373.15\,\mathrm{K}$ into Equation (2):

$$R(100\,^\circ\mathrm{C}) = 9835.46 \times \exp\left[3578.29 \times \left(\frac{1}{373.15} - \frac{1}{298.15}\right)\right] = 9835.46 \times e^{-2.412} = 881.4\,\Omega \tag{19}$$

Step 2b: Thermistor divider voltage at 100 °C.

$$V_{\mathrm{th}}(100\,^\circ\mathrm{C}) = 5.0 \times \frac{881.4}{10{,}000 + 881.4} = 0.4050\,\mathrm{V} \tag{20}$$

Step 2c: Difference amplifier output at 100 °C.

$$V_{\mathrm{out}}(100\,^\circ\mathrm{C}) = V_{\mathrm{ref}} - V_{\mathrm{th}}(100\,^\circ\mathrm{C}) = 3.730 - 0.405 = 3.325\,\mathrm{V} \tag{21}$$

This extrapolated value is adopted as $V_{\mathrm{AREF}} = 3.325\,\mathrm{V}$ on potentiometer 2, establishing the full ADC mapping:

- ADC $= 0 \longleftrightarrow V_{\mathrm{out}} = 0\,\mathrm{V} \longleftrightarrow 0\,^\circ\mathrm{C}$
- ADC $= 1023 \longleftrightarrow V_{\mathrm{out}} = 3.325\,\mathrm{V} \longleftrightarrow 100\,^\circ\mathrm{C}$ (nominal)

3.9.3. Voltage Table at All Calibration Points

Table 2 shows the thermistor divider voltage and expected difference amplifier output at all eight measured calibration points plus the 100 °C model extrapolation which defines AREF.

Table 2: Expected voltage levels at each calibration point. The final row (100 °C) is extrapolated from the fitted model and defines V_{AREF}.

Pt	T (°C)	R_{th} (Ω)	V_{th} (V)	$V_{\mathrm{out}} = V_{\mathrm{ref}} - V_{\mathrm{th}}$ (V)
1	0.0	29 498.05	3.7341	≈0.000
2	33.2	7 241.27	2.1000	1.630
3	34.4	6 832.37	2.0295	1.701
4	35.3	6 585.11	1.9852	1.745
5	37.0	6 135.16	1.9012	1.829
6	37.8	5 944.50	1.8641	1.866
7	38.2	5 892.11	1.8538	1.876
8	90.0	1 135.84	0.5100	3.220
100.0 (model)		*881.4*	*0.405*	*3.325* ← **AREF**

4. Circuit Design and Signal Conditioning

4.1. System Overview

The complete measurement chain consists of four stages connected in series. Stage 1 is a thermistor voltage divider converting resistance to voltage. Stage 2 is a reference potentiometer producing a fixed 3.73 V anchor. Stage 3 is a unity-gain difference amplifier subtracting the reference from the thermistor signal, so that 0 °C maps to 0 V output and 100 °C maps to 3.325 V output. Stage 4 applies 3.325 V to the Arduino AREF pin, mapping the full temperature range onto all 1023 ADC steps.

4.2. Stage 1: Thermistor Voltage Divider

4.2.1. Circuit Description

The fixed 10 kΩ series resistor (R_1) is connected between the Arduino 5 V rail and one lead of the thermistor. The other thermistor lead connects to the Arduino GND rail. The junction node between R_1 and the thermistor carries the signal voltage V_{th} described by Equation (6).

4.2.2. Wiring on the Breadboard

- **5 V rail → R_1 (10 kΩ):** One leg of the 10 kΩ resistor is inserted into a row connected to the breadboard positive power rail (5 V from Arduino).
- **R_1 output → thermistor input:** The other leg of R_1 and one lead of the thermistor share the same breadboard row — this is node V_{th}.
- **Thermistor output → GND rail:** The second thermistor lead connects to the breadboard GND rail.
- **V_{th} node → op-amp inverting input:** A short jumper wire links the V_{th} node to the row of the 1 MΩ input resistor ($R_{\text{in,inv}}$) of the difference amplifier in Stage 3.

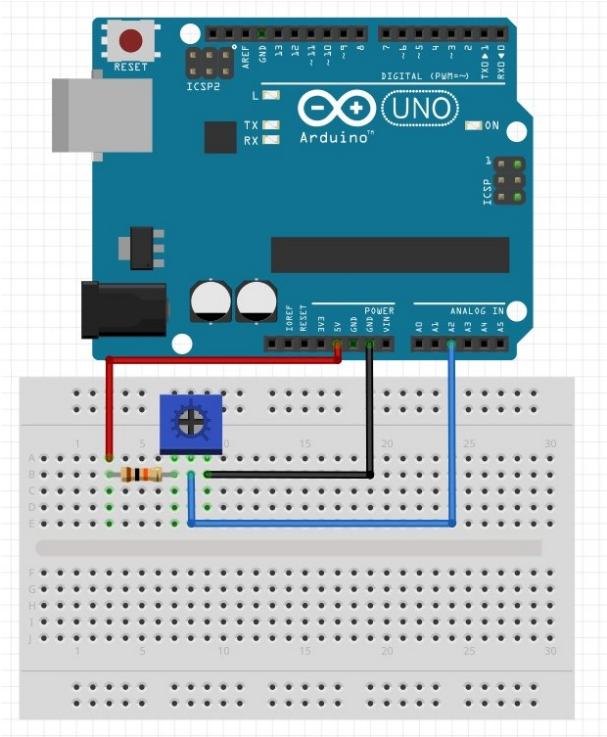


Figure 18: Stage 1 thermistor voltage divider wiring. The V_{th} node connects via jumper to the inverting input of Stage 3.

4.3. Stage 2: Reference Voltage Generator (3.73 V)

4.3.1. Circuit Description

A trimmer potentiometer is used to generate the stable 3.73 V reference voltage V_{ref} required by the difference amplifier. The potentiometer acts as an adjustable voltage divider (Equation (7)): with its two end terminals connected to 5 V and GND, the wiper can be adjusted to produce any voltage between 0 and 5 V.

4.3.2. Wiring on the Breadboard

- **Top terminal → 5 V rail:** Pin 1 (or pin 3, depending on potentiometer orientation) of the trimmer is wired to the breadboard positive rail.
- **Bottom terminal → GND rail:** The opposite end pin connects to the breadboard GND rail.
- **Wiper → op-amp non-inverting input:** The wiper (centre pin) is wired to the row of the 1 MΩ input resistor ($R_{\text{in,non}}$) on the non-inverting side of Stage 3.
- **Wiper → Arduino A2 (optional monitor):** A second jumper from the same wiper row to Arduino analog pin A2 allows the live voltage reader sketch (Appendix B) to display the wiper voltage in real time during adjustment.

4.3.3. Setting the Potentiometer with the Live Voltage Reader

A short Arduino sketch was run during circuit assembly to display the potentiometer wiper voltage at 2 Hz in the Serial Monitor. The wiper screw was turned slowly until the

reading converged to 3.73 V:

Listing 4: Core of the live voltage reader used to set both potentiometers.

```
float Vout  = analogRead(A2) * 5.0f / 1023.0f;
float R_pot = R_FIXED * (VREF / (VREF - Vout) - 1.0f);
Serial.print(Vout, 4);
Serial.print(",");
Serial.println(R_pot, 2);
```

The Serial Monitor output observed while converging to 3.73 V:

```
Vout[V], R_pot[Ohm]
4.4033,  58935.43
3.9345,  35002.55
3.7954,  28820.12
3.7354,  25701.34
3.7310,  25536.21
3.7300,  25491.80
3.7300,  25491.80   <- stable at target  3.730 V
```

Listing 5: Serial Monitor output during adjustment of potentiometer 1. Each line printed every 500 ms.

The same procedure was repeated for potentiometer 2 to set $V_{\mathrm{AREF}} = 3.325\,\mathrm{V}$ (Section 4.5).

4.4. Stage 3: Unity-Gain Difference Amplifier

4.4.1. Circuit Description and Pin Assignment

The op-amp IC (single-supply LM741 or equivalent) has its positive supply pin connected to the 5 V rail and its negative supply pin to GND. Four 1 MΩ resistors implement the difference amplifier topology shown in Figure 5:

- $R_{\mathbf{in,inv}}$ **(1 MΩ):** From the V_{th} node (Stage 1 output) to the op-amp inverting input (V−). This resistor carries the thermistor signal.
- R_F **(1 MΩ):** From the op-amp output back to V−. This is the feedback resistor that, together with $R_{\mathrm{in,inv}}$, sets the inverting gain to $-R_F/R_{\mathrm{in,inv}} = -1$.
- $R_{\mathbf{in,non}}$ **(1 MΩ):** From the potentiometer 1 wiper ($V_{\mathrm{ref}} = 3.73\,\mathrm{V}$) to the op-amp non-inverting input (V+).
- $R_{\mathbf{gnd}}$ **(1 MΩ):** From V+ to GND. This resistor is essential: it ensures both inputs see the same Thévenin source impedance, which is the condition for good common-mode rejection. Without it the gain would differ between the two input paths and the output would not be a clean subtraction.

4.4.2. Wiring on the Breadboard

- The op-amp straddles the breadboard centreline (pins of each side in separate half-columns so the IC body does not short-circuit connections across the gap).
- **Power:** Op-amp V_{CC+} pin → 5 V rail; op-amp V_{CC-} pin → GND rail.

- **Inverting path:** V_{th} row $\xrightarrow{R_{\text{in,inv}}}$ V− pin row $\xrightarrow{R_F}$ output pin row.
- **Non-inverting path:** Potentiometer 1 wiper row $\xrightarrow{R_{\text{in,non}}}$ V+ pin row $\xrightarrow{R_{\text{gnd}}}$ GND rail.
- **Output:** Op-amp output pin row → Arduino A0 via a short jumper wire. This is the input to the ADC.

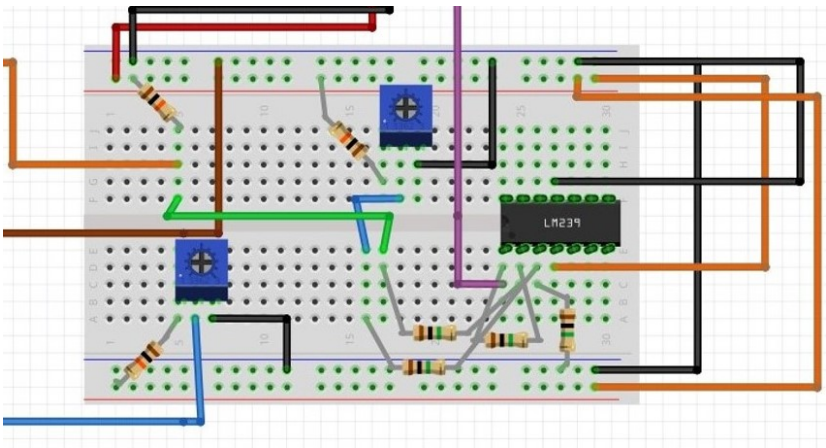


Figure 19: Stage 3 difference amplifier wiring. All four resistors are 1 MΩ. Inverting input: V_{th} from Stage 1. Non-inverting input: $V_{\text{ref}} = 3.73\,\text{V}$ from Stage 2. Output to Arduino A0.

4.4.3. Why 1 MΩ Resistors

The choice of 1 MΩ is driven entirely by the impedance matching requirement (Section 2.3.4). The Thévenin output impedance $R_{\text{Th}} = R_1 \| R_{\text{th}}$ varies with temperature; worst case is at 0 °C where $R_{\text{th}} = 29{,}498\,\Omega$:

$$R_{\text{Th,max}} = \frac{10{,}000 \times 29{,}498}{10{,}000 + 29{,}498} = 7{,}468\,\Omega \tag{22}$$

Table 3: Thévenin source impedance of the thermistor divider at each calibration temperature.

T (°C)	R_{th} (Ω)	$R_{\text{Th}} = R_1 \| R_{\text{th}}$ (Ω)
0.0	29 498	7 468
33.2	7 241	4 200
34.4	6 832	4 059
35.3	6 585	3 971
37.0	6 135	3 802
37.8	5 945	3 728
38.2	5 892	3 708
90.0	1 136	1 020

4.4.4. Verification of the 10× Impedance Rule

Applying Equation (10) at worst case:

$$10 \times R_{\text{Th,max}} = 10 \times 7{,}468 = 74{,}680\,\Omega \tag{23}$$

$$Z_{\text{in}} = 1{,}000{,}000\,\Omega \gg 74{,}680\,\Omega \quad \checkmark \quad (\text{margin factor} = 13.4) \tag{24}$$

4.4.5. Quantitative Loading Error at 0 °C

Using Equation (9):

$$\varepsilon_V = 3.7341 \times \frac{7{,}468}{1{,}000{,}000} = 27.9\,\text{mV} \tag{25}$$

Converting to temperature using the local divider sensitivity:

$$\left.\frac{dV_{\text{th}}}{dT}\right|_{0\,^\circ\text{C}} = V_{cc} \cdot \frac{R_1\,(dR/dT)}{(R_1 + R_{\text{th}})^2} = 5.0 \times \frac{10{,}000 \times (-1{,}415)}{(39{,}498)^2} = -45.3\,\text{mV/K} \tag{26}$$

$$\varepsilon_T = \frac{27.9\,\text{mV}}{45.3\,\text{mV/K}} \approx 0.62\,^\circ\text{C} \tag{27}$$

This is a fixed systematic bias. For comparison, with 100 kΩ resistors ε_V would be 279 mV (≈6 °C), and with 10 kΩ it would exceed 2.8 V, completely distorting the measurement.

4.4.6. Why the Output Was Not Amplified to 0–5 V

A natural alternative to the external AREF approach would be to set the difference amplifier gain to $G = 5.0/3.325 = 1.504$, stretching the output to the full 0–5 V ADC range and avoiding the AREF circuit entirely. This approach was considered and rejected for the following reasons:

1. **Single-supply output swing constraint.** The op-amp is powered from a single 5 V rail. Even rail-to-rail types cannot reach exactly 5 V — the output headroom is typically 50–100 mV below supply. An output of exactly 5.0 V is therefore unattainable regardless of gain setting.
2. **Non-standard resistor values.** $G = 1.504$ requires $R_F = 1.504\,R_{\text{in}}$. With $R_{\text{in}} = 1\,\text{M}\Omega$ this gives $R_F = 1{,}504\,\text{k}\Omega$ — not an E24 or E96 standard value. A series combination of two standard values would be required, adding wiring complexity and two additional sources of tolerance error.
3. **CMRR degradation with unequal resistors.** Common-mode rejection ratio depends on resistor matching. Any mismatch between the non-equal feedback and input pairs causes different errors at each input, partially defeating the differential measurement. The equal-resistor unity-gain configuration is inherently better balanced.
4. **AREF approach is simpler and equivalent.** Setting $V_{\text{AREF}} = 3.325\,\text{V}$ achieves the same goal — full utilisation of all 1023 ADC steps — without modifying the

amplifier stage at all. The second potentiometer uses no additional ICs and preserves the unity-gain equal-resistor difference amplifier.

4.5. Stage 4: External ADC Reference (3.325 V)

4.5.1. Circuit Description

A second trimmer potentiometer generates the $V_{\text{AREF}} = 3.325\,\text{V}$ external reference for the Arduino ADC, derived by querying the calibrated β-model at 100 °C (Equation (21)). Setting V_{AREF} equal to the model's predicted maximum diff-amp output at 100 °C ensures that ADC full scale (count 1023) corresponds exactly to the nominal upper measurement bound.

4.5.2. Wiring on the Breadboard

- **Top terminal → 5 V rail; Bottom terminal → GND rail:** Same as potentiometer 1.
- **Wiper → 5 kΩ protection resistor:** The wiper is connected in series with a 5 kΩ resistor before reaching the Arduino AREF pin. This resistor prevents damage if the firmware is accidentally compiled without `analogReference(EXTERNAL)` (which would connect the external source against the internal reference buffer inside the ADC). The 5 kΩ limits any fault current to a safe level.
- **5 kΩ output → Arduino AREF pin:** A jumper wire runs from the far end of the protection resistor directly to the AREF pin on the Arduino header.
- **Wiper → Arduino A1 (optional monitor):** A second jumper from the wiper node to A1 allows the live voltage reader to confirm the AREF voltage during adjustment.

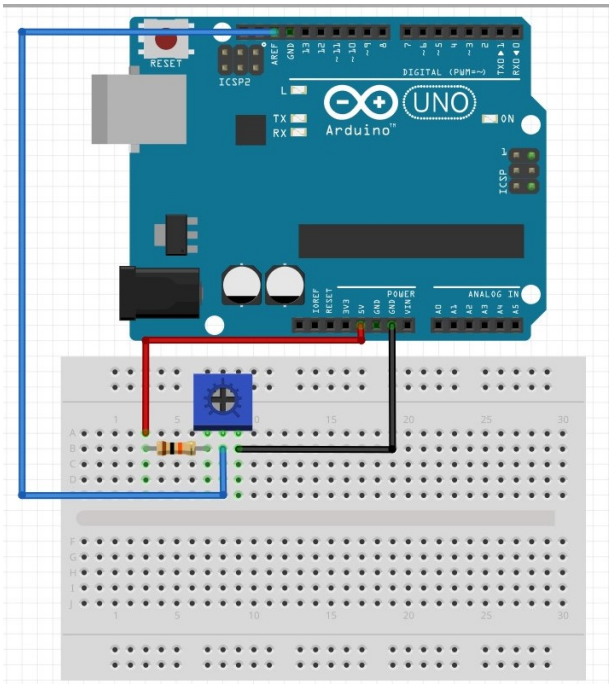


Figure 20: Stage 4 AREF wiring. The 5 kΩ protection resistor is mandatory between the potentiometer wiper and the AREF pin.

The resulting ADC voltage resolution is:

$$\Delta V = \frac{V_{\text{AREF}}}{1023} = \frac{3.325}{1023} = 3.25\,\text{mV/step} \tag{28}$$

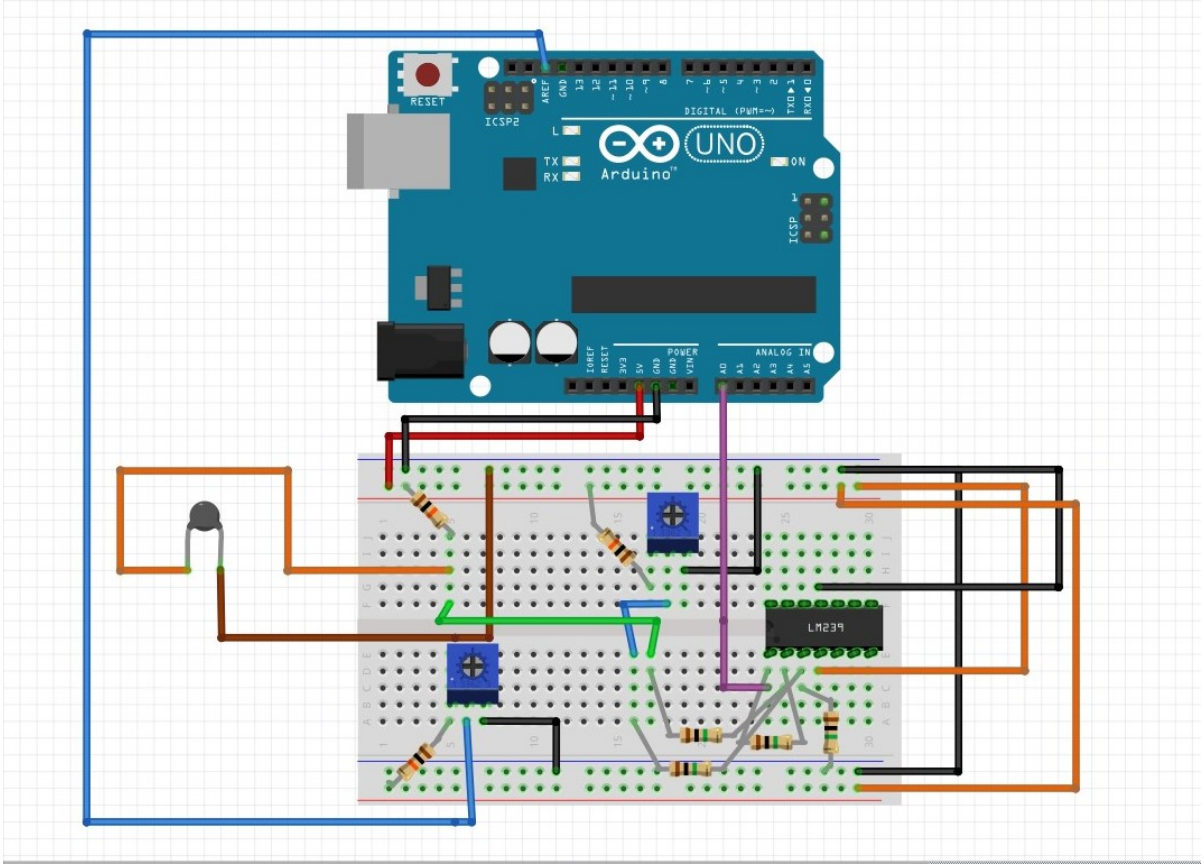
compared to 4.89 mV/step with the default 5 V reference.

4.6. Full Assembled Circuit

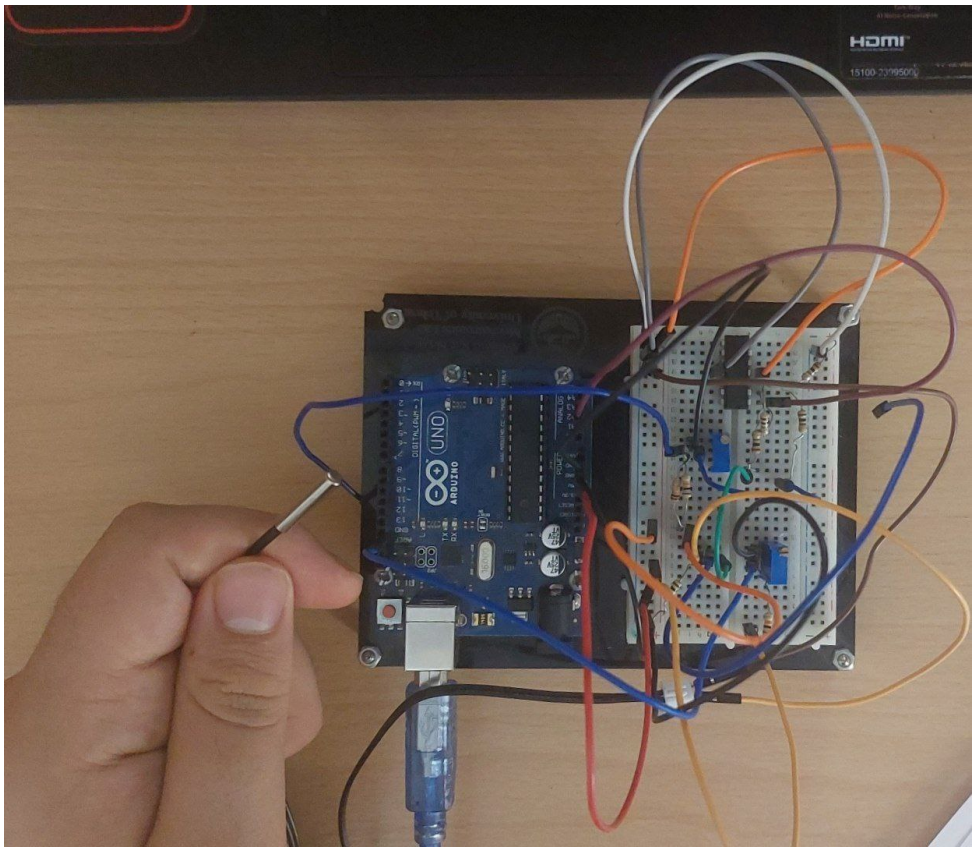
Figure 21 shows both the full hand-drawn circuit schematic and the corresponding assembled breadboard circuit side by side.

The schematic (Figure 21a) traces the complete signal path from the 5 V supply through the voltage divider to the thermistor, then through the op-amp with all four 1 MΩ resistors, and finally to the Arduino A0 input with the external AREF supply shown separately.

The physical breadboard layout (Figure 21b) implements this schematic left to right: the thermistor probe exits the board toward the measurement bath at the far left; the 10 kΩ series resistor and potentiometer 1 (3.73 V reference) occupy the left section; the op-amp IC sits at the centre with its four 1 MΩ resistors fanning outward; potentiometer 2 and the 5 kΩ AREF protection resistor occupy the right section; the Arduino Uno is connected at the far right.



(a) Full circuit schematic showing all four stages, component values, and net names.



(b) Fully assembled breadboard circuit.

Figure 21: Complete measurement circuit: schematic (left) and physical implementation (right). The stage-to-stage signal flow follows the schematic left-to-right order on the breadboard. A full walkthrough video is available at [4].

5. Firmware Implementation

5.1. AREF Safety Requirement

`analogReference(EXTERNAL)` **must be the very first call in** `setup()`, before any `analogRead()`. If an `analogRead()` is executed while the default internal reference is active and an external voltage is simultaneously present on the AREF pin, a short circuit is created between the external source and the internal reference buffer, permanently damaging the ADC. Calling `analogReference(EXTERNAL)` first disconnects the internal buffer before the ADC performs any conversion.

5.2. Signal-Chain Inversion Logic

The firmware reverses each stage of the conditioning chain in order:

// Stage 1 inversion: R = R1 * V$_t$h/(Vcc−V$_t$h)$floatcompute_Rtherm(floatv_therm)if(v_therm <= 0.01f$)

// Beta model: T(C) = 1/(1/T0 + ln(R/R0)/beta) - 273.15 float compute$_T$emperature($floatR_therm)fl$

The three-step chain is:

$$V_{\text{adc}} = \frac{\text{ADC}}{1023} \cdot V_{\text{AREF}} \xrightarrow{\text{undo Stage 3}} V_{\text{th}} = V_{\text{ref}} - V_{\text{adc}} \xrightarrow{\text{undo Stage 1}} R_{\text{th}} = R_1 \cdot \frac{V_{\text{th}}}{V_{cc} - V_{\text{th}}} \xrightarrow{\beta\text{-model}} T \tag{29}$$

5.3. Firmware Constants and Their Origins

Every constant hard-coded in the firmware is traceable to a specific calibration step:

Table 4: Firmware constants and their derivation sources.

Constant	Value	Type	Source
`V_AREF`	3.325 V	Derived	Model extrap. at 100 °C (Eq. (21))
`V_FIXED_REF`	3.730 V	Measured	Ice bath SS mean (Fig. 8)
`R_SERIES`	10 000 Ω	Nominal	Fixed divider resistor
`VCC`	5.0 V	Nominal	Arduino 5 V rail
`R0`	9835.46 Ω	Fitted	MATLAB `polyfit` (Eq. (16))
`BETA`	3578.29 K	Fitted	MATLAB `polyfit` (Eq. (16))
`T0_K`	298.15 K	Fixed	Standard reference (25 °C)

6. Results and Verification

6.1. Verification Procedure

After uploading the final firmware the system was verified at two points:

1. **Ice bath (0 °C):** The thermistor probe was re-immersed in the same ice bath used for calibration. The Serial Monitor was observed until the reported temperature stabilised. A correctly calibrated system should display $0 \pm \varepsilon_T$ °C, where ε_T is the expected loading error of ≈0.62 °C (Equation (27)).
2. **Mixed bath with thermometer (34.4 °C):** The probe was co-immersed with the medical thermometer in a stable mixed bath. The firmware output was compared directly to the thermometer reading.

6.2. Verification Results — Thermometer Agreement

Ice bath verification:

```
V_adc,   V_th,    R_th,        Temp_C
0.0161,  3.7139,  28836.59,    0.44
0.0161,  3.7139,  28836.59,    0.44
0.0163,  3.7137,  28824.11,    0.44
0.0161,  3.7139,  28836.59,    0.44
```

Listing 6: Serial Monitor output at the ice bath during final verification. System reports +0.44 °C rather than the ideal 0.00 °C, consistent with the predicted loading offset of 0.62 °C.

The residual +0.44 °C is within the predicted systematic loading error of 0.62 °C and is also partly attributable to the op-amp output floor (the single-supply LM741 cannot swing to true 0 V).

Mixed bath verification at 34.4 °C:

```
V_adc,   V_th,    R_th,        Temp_C
1.6884,  2.0416,  6922.11,     34.32
1.6884,  2.0416,  6922.11,     34.32
1.6933,  2.0367,  6887.82,     34.38
1.6884,  2.0416,  6922.11,     34.32
```

Listing 7: Serial Monitor output at the 34.4 °C mixed bath. The firmware reports 34.32 °C against a medical thermometer reading of 34.4 °C — a discrepancy of only 0.08 °C.

6.3. Summary of Verification Accuracy

Table 5: Final system verification results.

Test	**Reference** (°C)	**Firmware** (°C)	**Error** (°C)	**Reference type**
Ice bath	0.00	+0.44	+0.44	Thermodynamic fixed point
Mixed bath	34.4	34.32	−0.08	Medical thermometer

7. Discussion

7.1. Model Validity and Calibration Density

The β-parameter model is an approximation valid over moderate temperature ranges; it captures the dominant exponential behaviour of NTC thermistors but neglects higher-order terms that become significant over ranges exceeding $\sim$100 °C. Within the 0–100 °C range targeted here the two-parameter model is well-established as adequate [1].

A more significant concern is the **non-uniform distribution of calibration points**. Six of the eight points lie in a 5 °C window (33–38 °C) and contribute very little independent information: their linearised abscissa values $x_i = 1/T_i - 1/T_0$ differ by only $\approx 5 \times 10^{-5}\,\mathrm{K}^{-1}$. The model parameters β and R_0 are therefore almost entirely determined by the leverage of the two anchor points at 0 °C and 90 °C. This is not detrimental to accuracy at those anchors, but it means the intermediate-temperature regime is interpolated with limited independent confirmation. A future improvement would be to collect additional points at 10, 20, 50, and 70 °C using a thermocouple reference, which would both over-determine the fit and allow residual analysis.

7.2. ADC Resolution Improvement

Table 6: ADC utilisation: default 5 V reference vs. calibrated $V_{\mathrm{AREF}} = 3.325\,\mathrm{V}$ external reference.

Parameter	**Default (5 V)**	**External (3.325 V)**
AREF voltage	5.000 V	3.325 V
Signal span (0–100 °C)	0–3.325 V	0–3.325 V
ADC steps utilised	680 / 1023 (66.5%)	1023 / 1023 (100%)
Voltage per step	4.89 mV	3.25 mV
Voltage resolution improvement	—	+33.5%
Additional steps gained	—	+343 steps

7.3. Error Sources and Analysis

1. **Systematic loading error (0.62 °C at 0 °C).** Calculated in Equation (27): the 27.9 mV offset produced by the finite 1 MΩ input impedance is a fixed bias that cannot be removed by averaging. An instrumentation amplifier (Section 2.6) would eliminate it entirely.
2. **Low calibration point count and uneven distribution.** Discussed in Section 7.1. Six of eight points concentrated in a 5 °C band limits model robustness outside that range. More calibration points spread over the full 0–100 °C range would significantly reduce systematic model error.
3. **Potentiometer setting accuracy.** Both V_{ref} and V_{AREF} were set manually using the live voltage reader with the Arduino's own 10-bit ADC ($\Delta V \approx 4.89$ mV resolution at 5 V). A residual 10 mV setting error in V_{ref} propagates directly as a 10 mV offset in every reconstructed V_{th}, corresponding to ≈0.22 °C at 25 °C. A precision multimeter used during adjustment reduces this error substantially.
4. **Medical thermometer accuracy for intermediate points.** Clinical thermometers are calibrated for 35–42 °C and may carry ±0.3–0.5 °C systematic error near the edges of their valid range. The ice bath is immune to this, being a thermodynamic fixed point.
5. **ADC quantisation.** At 3.25 mV/step and $|dV_{\text{th}}/dT| \approx 45.3$ mV/K at 0 °C (Equation (26)), the quantisation-limited temperature resolution is $3.25/45.3 \approx 0.07$ °C near 0 °C.
6. **Thermistor self-heating.** The 5 V supply drives ≈250 μA through the thermistor at 25 °C, dissipating ≈0.6 mW and producing a small positive temperature bias in still air. In a well-stirred liquid bath this error is negligible due to the large convective heat transfer coefficient.
7. **Op-amp output floor on single supply.** On a 5 V single-supply the op-amp cannot swing to true 0 V at its output. The finite floor (∼50–100 mV) partially explains the +0.44 °C residual at the ice bath seen in Listing 6.
8. **Model extrapolation uncertainty.** The ADC reference was derived by querying the model at 100 °C, which lies 10 °C beyond the highest directly measured point (90 °C). Over such a short extrapolation the β-model is well-behaved, but any systematic error in β is amplified slightly compared to an interpolated prediction. This uncertainty could be eliminated by directly measuring at 100 °C using a thermocouple reference.

8. Conclusion

A complete NTC thermistor temperature measurement system was designed, calibrated, and verified as a functioning thermometer. The calibration strategy was constrained by the available reference instrument: a medical thermometer with a valid range of 32–43 °C, which forced all intermediate calibration points into a narrow body-temperature cluster. The 0 °C and 90 °C anchor points were collected using instrument-independent references — ice bath and boiling water — with the 90 °C assignment physically justified by Tehran's altitude (≈1200 m) and the rapid Newton's law cooling observed during the 60-second acquisition (Figure 15).

The ice bath steady-state reading ($V_{\text{th}} = 3.73\,\text{V}$, Figure 8) determined the difference amplifier reference V_{ref}. Rather than using the direct 90 °C measurement as the AREF upper bound, the fitted β-model was queried at 100 °C to extrapolate the expected maximum diff-amp output (Equation (21)):

$$V_{\text{AREF}} = V_{\text{ref}} - V_{\text{th}}(100\,^\circ\text{C}) = 3.730 - 0.405 = 3.325\,\text{V}$$

This maps ADC full scale (count 1023) to the nominal 100 °C upper bound rather than the measured 90 °C, achieving a voltage resolution of 3.25 mV/step and a 33.5% improvement over the 5 V default. An external gain stage to stretch the output to 0–5 V was deliberately avoided: the single-supply op-amp cannot reach 5 V, non-standard resistor values would degrade CMRR, and the potentiometer AREF approach achieves identical ADC utilisation with a simpler, equal-resistor amplifier topology (Section 4.4.6).

The 1 MΩ difference amplifier resistors satisfy the 10× impedance rule by a factor of 13.4 (Equation (24)), yielding a worst-case loading error of 0.62 °C at 0 °C. System verification (Section 6.2) returned +0.44 °C at the ice bath (Listing 6) and 34.32 °C against a medical thermometer reading of 34.4 °C (Listing 7) — a discrepancy of only 0.08 °C, confirming that the system meets its design accuracy target.

Acknowledgements

The authors gratefully acknowledge the guidance and support received throughout this course. We would like to express our sincere gratitude to **Dr. Ali Sadighi** for designing the course structure, providing the theoretical framework required in the laboratory, and offering clear instruction throughout the semester.

We also thank the teaching assistants, **Mohammad Jadidi** and **Erfun Bayat Sarmadi**, for their patient assistance and guidance throughout this semester.

References

[1] M. Robati, A. Mashhadireza, S. Sehhat, M. Jadidi, E. B. Sarmadi, "Lab Session 06: Temperature Measurement System," *Mechatronics Laboratory Manual*, University of Tehran, ver. 1.2, Dec. 2025.

[2] D. G. Alciatore, *Introduction to Mechatronics and Measurement Systems*, 4th ed., McGraw-Hill, 2012.

[3] Analog Devices, "AD620 Low Cost Instrumentation Amplifier," Data Sheet. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/AD620.pdf

[4] Roozbeh Amini, "NTC Thermistor Temperature Measurement System — Circuit Walkthrough and Experimental Demonstration," *Supplementary Video, Lab Session 06*, University of Tehran, Feb. 2026. [Online]. Available: Google Drive (click to open video)

Appendices

A. Arduino Calibration Logger (50 Hz, 60 s)

Listing 8: Full 50 Hz calibration logger. Streams `t_s,adc,volt_V,R_therm_Ohm` as CSV for 60 s, then halts permanently via `static bool done`.

```
const int             THERM_PIN     = A0;
const float           VREF          = 5.0f;
const float           R_FIXED       = 10000.0f;
const unsigned long   DURATION_MS   = 60000UL;
const unsigned long   Ts_ms         = 20UL;      // 50 Hz

void setup() {
  Serial.begin(115200);
  Serial.println("t_s,adc,volt_V,R_therm_Ohm");
}

void loop() {
  static bool done = false;
  if (done) return;

  unsigned long t0   = millis();
  unsigned long next = t0;

  while (millis() - t0 < DURATION_MS) {
    if (millis() >= next) {
      next += Ts_ms;
      float t_s  = (millis() - t0) / 1000.0f;
      int   adc  = analogRead(THERM_PIN);
      float v    = adc * VREF / 1023.0f;
      float Rth  = (v > 0.01f && v < VREF - 0.01f)
                     ? R_FIXED * v / (VREF - v) : 0.0f;
      Serial.print(t_s,  4); Serial.print(",");
      Serial.print(adc);     Serial.print(",");
      Serial.print(v,    6); Serial.print(",");
      Serial.println(Rth, 2);
    }
  }
  done = true;
}
```

B. Live Voltage and Resistance Reader

Listing 9: Full live voltage reader used to set both potentiometers. Prints `Vout[V],R_pot[Ohm]` every 500 ms.

```
const int   PIN_ANALOG = A2;
const float VREF       = 5.0f;
const int   ADC_RES    = 1023;
const float R_FIXED    = 10000.0f;

void setup() {
  analogReference(DEFAULT);
```

```
  Serial.begin(9600);
  while (!Serial) { ; }
  Serial.println("Vout[V],R_pot[Ohm]");
}

void loop() {
  int   adc   = analogRead(PIN_ANALOG);
  float Vout  = (float)adc * VREF / (float)ADC_RES;
  float R_pot = -1.0f;
  if (Vout > 0.01f)
    R_pot = R_FIXED * (VREF / (VREF - Vout) - 1.0f);
  Serial.print(Vout,  4); Serial.print(",");
  Serial.println(R_pot, 2);
  delay(500);
}
```

C. MATLAB Calibration Acquisition Script

Listing 10: Full MATLAB acquisition: opens serial port, collects 60 s, extracts steady-state window, generates annotated plots, and appends to the master calibration CSV.

```
clear; clc; close all;
port       = "COM4";
baud       = 115200;
duration   = 60;
steady_win = 10;

ref_temp = input('>> Enter bath temperature (deg C): ');

s = serialport(port, baud);
configureTerminator(s, "LF");
flush(s); pause(1);
if s.NumBytesAvailable > 0, readline(s); end

count = 0;
buf   = zeros(duration * 60, 3);
tic;
while toc < duration
  if s.NumBytesAvailable > 0
    vals = sscanf(readline(s), "%f,%d,%f,%f");
    if numel(vals) == 4
      count = count + 1;
      buf(count, :) = [vals(1), vals(3), vals(4)];
    end
  end
end
delete(s); clear s;

T = buf(1:count, 1);
V = buf(1:count, 2);
R = buf(1:count, 3);

%% Steady-state extraction
t_max  = max(T);
ss     = T >= (t_max - steady_win);
v_mean = mean(V(ss));
```

```
r_mean = mean(R(ss));
r_std  = std(R(ss));

%% Save full time series
tag      = strrep(sprintf('%.1f', ref_temp), '.', 'p');
raw_file = sprintf('cal_raw_%sC.csv', tag);
fid = fopen(raw_file, 'w');
fprintf(fid, 't_s,volt_V,R_therm_Ohm\n');
for k = 1:count
  fprintf(fid, '%.4f,%.6f,%.2f\n', T(k), V(k), R(k));
end
fclose(fid);

%% Append to master list
master = 'calibration_master_list.csv';
if ~isfile(master)
  fid = fopen(master, 'w');
  fprintf(fid, 'T_C,V_mean,R_mean\n');
  fclose(fid);
end
fid = fopen(master, 'a');
fprintf(fid, '%.2f,%.6f,%.6f\n', ref_temp, v_mean, r_mean);
fclose(fid);

%% Annotated plots
figure('Color','w','Position',[100 100 1200 500]);
subplot(2,1,1);
plot(T,V,'Color',[.75 .75 .75]); hold on;
plot(T,movmean(V,50),'b','LineWidth',1.8);
yl=ylim;
patch([t_max-steady_win t_max t_max t_max-steady_win],...
      [yl(1) yl(1) yl(2) yl(2)],[0.18 0.78 0.18],...
      'FaceAlpha',.12,'EdgeColor','none');
yline(v_mean,'r--','LineWidth',1.6,...
  'Label',sprintf('V_{ss}=%.4f V',v_mean),...
  'LabelHorizontalAlignment','left','LabelVerticalAlignment','bottom');
ylabel('V_{th} (V)'); grid on; box on;
legend('Raw 50 Hz','Smoothed 1 s','Steady window','Location','best');
title(sprintf('Voltage trace at %.1f degC', ref_temp));

subplot(2,1,2);
plot(T,R,'Color',[.85 .33 .10],'LineWidth',1.4); hold on;
yl2=ylim;
patch([t_max-steady_win t_max t_max t_max-steady_win],...
      [yl2(1) yl2(1) yl2(2) yl2(2)],[0.18 0.78 0.18],...
      'FaceAlpha',.12,'EdgeColor','none');
yline(r_mean,'k--','LineWidth',1.6,...
  'Label',sprintf('R_{ss}=%.2f Ohm',r_mean),...
  'LabelHorizontalAlignment','left','LabelVerticalAlignment','bottom');
xlabel('Time (s)'); ylabel('R_{th} (Ohm)'); grid on; box on;
title(sprintf('Resistance trace at %.1f degC', ref_temp));
sgtitle(sprintf('SS: V=%.4fV | R=%.2fOhm | sigma=%.3fOhm',...
        v_mean,r_mean,r_std),'FontWeight','bold');

fprintf('Done: %.1fC | %.4fV | %.2fOhm\n', ref_temp, v_mean, r_mean);
```

D. MATLAB Model Fitting and Characteristic Plots

Listing 11: MATLAB model fitting: loads the master calibration CSV, fits the linearised β model, generates the calibration curve and expected characteristic plots, extrapolates to 100 °C to determine V_{AREF}, and saves all figures to the Desktop.

```
clear; clc; close all;

%% Setup
if ispc
  desktop = fullfile(getenv('USERPROFILE'),'Desktop');
else
  desktop = fullfile(getenv('HOME'),'Desktop');
end
save_dir = fullfile(desktop,'ThermistorFigures');
if ~exist(save_dir,'dir'), mkdir(save_dir); end
DPI = 300;

%% Load data
csv_path = fullfile(desktop,'calibration_master_list.csv');
if isfile(csv_path)
  data = readmatrix(csv_path);
  T_C  = data(:,1);  R_meas = data(:,3);
else
  T_C    = [0.0;33.2;34.4;35.3;37.0;37.8;38.2;90.0];
  R_meas = [29498.05;7241.27;6832.37;6585.11;...
            6135.16;5944.50;5892.11;1135.84];
end
[T_C,ix] = sort(T_C);
R_meas   = R_meas(ix);
T_K      = T_C + 273.15;
T0       = 298.15;
Vcc = 5.0;  R1 = 10000;  Vref = 3.730;

%% Beta-model fit
X    = (1./T_K) - (1/T0);
Y    = log(R_meas);
p    = polyfit(X, Y, 1);
Beta = p(1);
R0   = exp(p(2));
fprintf('Beta = %.4f K\nR0   = %.4f Ohm\n', Beta, R0);

%% Extrapolate to 100 degC
T_extrap = 100;
R_100    = R0 * exp(Beta * (1/(T_extrap+273.15) - 1/T0));
Vth_100  = Vcc * R_100 / (R1 + R_100);
Vout_100 = Vref - Vth_100;
fprintf('\nAt 100 degC (model extrapolation):\n');
fprintf('  R(100) = %.2f Ohm\n', R_100);
fprintf('  Vth(100) = %.4f V\n', Vth_100);
fprintf('  Vout(100) = %.4f V --> VAREF = %.3f V\n', Vout_100, Vout_100)
    ;

%% Model curve
Tc   = linspace(0, 105, 600)';
Tk   = Tc + 273.15;
Xp   = (1./Tk) - (1/T0);
Rm   = R0 .* exp(Beta .* Xp);
```

```
Vth_model  = Vcc .* Rm   ./ (R1 + Rm);
Vout_model = Vref - Vth_model;
Vth_data   = Vcc .* R_meas ./ (R1 + R_meas);
Vout_data  = Vref - Vth_data;

%% --- FIGURE 1: Calibration Curve (no CI) ---
hf1 = figure('Color','w','Position',[50 50 1000 560],'Visible','on');
ax1 = axes(hf1); hold on;

h_fit  = plot(Tc, Rm, 'b-', 'LineWidth', 2.2);
h_data = scatter(T_C, R_meas, 80, 'r', 'filled', ...
                 'MarkerEdgeColor',[0.55 0 0],'LineWidth',1.2);

h_ext = scatter(T_extrap, R_100, 100, [0 0.6 0], 'd', 'filled', ...
                'MarkerEdgeColor',[0 0.35 0],'LineWidth',1.5);
text(T_extrap+1.5, R_100+300, ...
     sprintf('100 degC (model)\nR = %.0f Ohm', R_100), ...
     'FontSize', 9, 'Color', [0 0.45 0]);

idx0  = find(T_C == 0,  1);
idx90 = find(T_C == 90, 1);
text(1, R_meas(idx0)-2200, ...
     sprintf('0 degC\n%.0f Ohm', R_meas(idx0)), ...
     'FontSize', 9, 'Color', [0.5 0 0]);
text(83, R_meas(idx90)+500, ...
     sprintf('90 degC  %.0f Ohm', R_meas(idx90)), ...
     'FontSize', 9, 'Color', [0.5 0 0]);

xregion(33, 38.5, 'FaceColor',[1 1 0],'FaceAlpha',0.15, ...
        'HandleVisibility','off');
text(35.5, 18000, '33-38 degC cluster', ...
     'HorizontalAlignment','center','FontSize',9,'Color',[0.5 0.4 0]);

xlabel('Temperature (degC)', 'FontSize',12);
ylabel('Thermistor Resistance (Ohm)', 'FontSize',12);
title(sprintf('Calibration Curve   Beta = %.2f K,   R0 = %.2f Ohm',...
      Beta, R0), 'FontSize',13,'FontWeight','bold');
legend([h_fit, h_data, h_ext], ...
       'Fitted beta-model', ...
       'Calibration points', ...
       '100 degC extrapolation (AREF basis)', ...
       'Location','northeast','FontSize',10);
grid on; box on;
set(gca,'FontSize',10);

exportgraphics(hf1, fullfile(save_dir,'fig_fitted_curve.png'), ...
               'Resolution', DPI);
fprintf('Saved: fig_fitted_curve.png\n');
close(hf1);

%% --- FIGURE 2: Expected R(T) and V_th(T) ---
hf2 = figure('Color','w','Position',[50 50 1200 500],'Visible','on');

subplot(1,2,1); hold on;
plot(Tc, Rm, 'b-', 'LineWidth', 2.2);
scatter(T_C, R_meas, 65, 'r', 'filled', ...
        'MarkerEdgeColor','k');
xlabel('Temperature (degC)','FontSize',12);
```

```
ylabel('Resistance (Ohm)','FontSize',12);
title('Expected R(T)','FontSize',12,'FontWeight','bold');
legend('Fitted model','Calibration data','Location','northeast','
    FontSize',10);
grid on; box on; set(gca,'FontSize',10);

subplot(1,2,2); hold on;
plot(Tc, Vth_model, 'r-', 'LineWidth', 2.2);
scatter(T_C, Vth_data, 65, 'k', 'filled');
text(1.5,  Vth_data(T_C==0)+0.06,  ...
     sprintf('3.73 V  (0 degC)'),  'FontSize',9,'Color',[0 0.4 0]);
text(72, Vth_data(T_C==90)+0.06, ...
     sprintf('0.51 V  (90 degC)'), 'FontSize',9,'Color',[0 0.4 0]);
scatter(T_extrap, Vth_100, 80, [0 0.6 0], 'd', 'filled', ...
        'MarkerEdgeColor',[0 0.35 0]);
text(T_extrap-18, Vth_100-0.06, ...
     sprintf('0.405 V  (100 degC model)'), 'FontSize',9,'Color',[0 0.4
        0]);
xlabel('Temperature (degC)','FontSize',12);
ylabel('V_{th}  (V)','FontSize',12);
title('Expected V_{th}(T)','FontSize',12,'FontWeight','bold');
legend('Fitted model','Calibration data','Location','northeast','
    FontSize',10);
grid on; box on; set(gca,'FontSize',10);

sgtitle(sprintf('Expected Thermistor Characteristics   Beta = %.2f K,
    R0 = %.2f Ohm',...
        Beta, R0), 'FontSize',12,'FontWeight','bold');

exportgraphics(hf2, fullfile(save_dir,'fig_expected_curves.png'), ...
               'Resolution', DPI);
fprintf('Saved: fig_expected_curves.png\n');
close(hf2);
```

E. Final Temperature Measurement Firmware

Listing 12: Complete final firmware. `analogReference(EXTERNAL)` is the mandatory first call. Reports V_{adc}, V_{th}, R_{th}, and T at 2 Hz. All calibration constants are taken directly from the MATLAB fit (Appendix D). $V_{\text{AREF}} = 3.325\,\text{V}$ is derived from the model extrapolation to 100 °C (Eq. (21)).

```
/*
 * Final Temperature Measurement Firmware
 * Lab Session 06 -- Mechatronics Lab, University of Tehran
 *
 * Forward signal chain:
 *   5V --[10k]--+--[Thermistor]-- GND
 *               |
 *              Vth   -->  Diff-Amp (V-)
 *   Pot-1 (3.730V)   -->  Diff-Amp (V+)
 *   Diff-Amp output  -->  A0
 *   Pot-2 (3.325V)   -->  [5k] --> AREF
 *
 * Firmware inversion:
 *   V_adc  = (ADC/1023) * V_AREF          (undo ADC scaling)
 *   V_th   = V_FIXED_REF - V_adc          (undo Stage 3 diff amp)
```

```
 *    R_th    = R_SERIES * Vth/(Vcc-Vth)    (undo Stage 1 divider)
 *    T[C]    = 1/(1/T0 + ln(R/R0)/beta) - 273.15
 *
 * V_AREF = 3.325 V derived from beta-model at 100 C:
 *    R(100C) = 881.4 Ohm -> Vth(100C) = 0.405 V
 *    Vout(100C) = 3.730 - 0.405 = 3.325 V -> AREF
 */

// --- Calibration constants (from MATLAB beta-model fit) ---
const float V_AREF      = 3.325f;   // model extrapolation to 100 C
const float V_FIXED_REF = 3.730f;   // Pot-1: V_th at 0 C (ice bath)
const float R_SERIES    = 10000.0f;
const float VCC         = 5.0f;
const float R0          = 9835.46f;
const float BETA        = 3578.29f;
const float T0_K        = 298.15f;

const int SENSOR_PIN = A0;

float reconstruct_Vtherm(float v_adc);
float compute_Rtherm(float v_therm);
float compute_Temperature(float R_therm);

void setup() {
  analogReference(EXTERNAL);   // MUST be first
  Serial.begin(9600);
  while (!Serial) { ; }
  Serial.println("V_adc,  V_th,    R_th,       Temp_C");
}

void loop() {
  int   adc    = analogRead(SENSOR_PIN);
  float v_adc  = (float)adc / 1023.0f * V_AREF;
  float v_therm = reconstruct_Vtherm(v_adc);
  float R_therm = compute_Rtherm(v_therm);
  float tempC  = compute_Temperature(R_therm);

  Serial.print(v_adc,  4); Serial.print(", ");
  Serial.print(v_therm, 4); Serial.print(", ");
  Serial.print(R_therm, 2); Serial.print(", ");
  Serial.println(tempC, 2);

  delay(500);   // 2 Hz
}

float reconstruct_Vtherm(float v_adc) {
  return V_FIXED_REF - v_adc;
}

float compute_Rtherm(float v_therm) {
  if (v_therm <= 0.01f) v_therm = 0.01f;
  if (v_therm >= 4.99f) v_therm = 4.99f;
  return R_SERIES * (v_therm / (VCC - v_therm));
}

float compute_Temperature(float R_therm) {
  float inv_T = (1.0f / T0_K) + (logf(R_therm / R0) / BETA);
  return (1.0f / inv_T) - 273.15f;
```

```
74 }
```